ENGINEERING SPECIFICATION · V7.1

3-Tier Platform Plan

Global → Country → City / Chapter

A complete roadmap for evolving the AI Salon platform from a single-chapter deployment into a fully tiered multi-tenant architecture. Covers schema completion, permission enforcement, brand-asset inheritance, email-system tiering, timezone de-hardcoding, route-level scope enforcement, and a phased migration plan. All 9 design decisions resolved.

7	22+	2	32-47
Migration phases	Models touched	New models	Engineer-days

Prepared by MassaPro team

Document Status

Status: All 9 design decisions resolved. Ready for Phase 0 implementation.

This document is the binding engineering specification for the AI Salon V7.1 3-tier platform migration. All open questions from the original planning round have been resolved by the MassaPro team and are recorded in Section 12. Implementation can proceed in the phased sequence defined in Section 10.

The 9 decisions cover: (1) chapter-prefixed URL strategy, (2) chapter admin isolation policy, (3) cross-chapter event visibility, (4) country-tier email template inheritance, (5) Super Admin allowlist management, (6) i18n scope, (7) member auto-chapter on RSVP, (8) Media Library architecture, and (9) cleanup phase timing. Each decision includes its rationale and the specific phase(s) it impacts.

An Implementation Feasibility Addendum (IMPL-1) follows the main plan. It stress-tests every section against the actual V7 codebase and corrects three effort estimates that were too low in the original plan (timezone migration 5-10× bigger, admin API scope enforcement 30+ routes vs. 2, EmailQueue write-path gaps requiring Phase 0 hotfix).

Table of Contents

3-Tier Completion Plan — Global → Country → City/Chapter

Scope of this plan. V7 shipped the schema, the 3-role model, the scope helpers, and per-chapter brand-image overrides for `favicon` / `loginHero` / `loginBanner`. What V7 did NOT ship: tier-coverage for QuizSession / Testimonial / ChatRoom / EventMockupDefault / EventImage / PresentationFile / SpeakerMessage / ConversationMessage / MemberTag / EventPrep* / EmailEvent / TrackingLog / EmailFlowStep / Quiz*; country-tier brand inheritance (Country has email defaults but NO logo/hero/banner/favicon columns); the `ChapterEmailTemplateOverride` admin UI; consumption of `Country.defaultEmailDomain/defaultFromName/defaultReplyTo` by the email sender; wiring of `relay-recipients.ts` into the speaker-message and DM relay paths; a "current scope" context switcher in the admin UI; per-chapter timezone propagation (still hard-coded `Asia/Jerusalem` in ~15 files); per-chapter UI copy (still hard-coded `AI Salon Tel Aviv` in ~30 files); a tier-aware Media Library; chapter-prefixed public URLs (or an explicit decision to keep flat URLs). This plan completes all of it.

Section 0 — Executive Summary

Current state — what V7 already delivered (in production):

1. **Schema + migration applied** (`prisma/migrations/20260719000000_v7_add_hierarchy/migration.sql`). Four new models: `Country`, `Chapter`, `ChapterSetting`, `ChapterEmailTemplateOverride`. `chapterId` added (nullable, indexed, FK → `Chapter(id)` ON DELETE SET NULL) to 13 models: `User`, `Event`, `Speaker`, `EventRsvp`, `EmailQueue`, `EmailRecipient`, `EmailCampaign`, `EmailTemplate`, `EmailStageTemplate`, `EmailFlow`, `EmailAudience`, `ReferralVisit`, `ReferralAttribution`. `Event` also got `isCrossChapter Boolean` for cross-chapter events (country-wide visibility).
2. **Seed endpoint** `/api/admin/v7-seed` (Super Admin only, idempotent) creates `Country(israel)` + `Chapter(tel-aviv)` and backfills ALL NULL `chapterId` / `countryId` rows to Israel/Tel Aviv. Already run on production.
3. **Scope helpers** in `src/lib/permissions.ts`: `getUserScope(userId)` → `{kind:"global"|"country"|"chapter"|"none"}`; `scopeUserWhere/scopeEventWhere/scopeChapterWhere(scope)` build Prisma `where` fragments; `canActOnChapter/canActOnCountry(scope, id)` for write-side checks; `getManagedChapterIds(userId, role)` returns `null|[ids]`. Production code uses these (not the dead `v7-scope.ts` duplicates).
4. **Admin pages already scoped** (server-side): `/admin` (members + recent events), `/admin/events`, `/admin/registrants`, `/admin/speakers`, `/admin/email` (campaigns/templates/flows/audiences/stageTemplates via `emailModelWhere` clause), `/admin/analytics`, `/admin/chapters`, `/admin/countries`, `/admin/reports`, `/admin/dashboard`. Each renders a colored scope badge in the header.
5. **Public chapter landing** `/c/[chapterSlug]` exists with hero image (`Chapter.heroImageUrl`), WhatsApp + LinkedIn links, member count, upcoming events list. New-user signup from `/c/[chapterSlug]` auto-tags the user with the chapter's `countryId` + `chapterId`.

6. **Per-chapter brand-image overrides** (favicon, loginHero, loginBanner) work end-to-end: admin uploads via `/admin/images` (with country+chapter filter); resolver `getEffectiveBrandImages(chapterId)` merges chapter overrides over `SiteSetting` global; `/login?chapterSlug=` and `/c/[chapterSlug]` generateMetadata consume it.
7. **Super Admin can allocate scope** via EditMemberDialog: country + chapter dropdowns + live "effective scope" preview; `PATCH /api/admin/members/[id]` validates `chapter.countryId === countryId`.
8. **V7 scope badge** on every admin page (purple=global, pink=country, cyan=chapter).

Target state — the completed 3-tier platform:

- Every content type the user listed (emails, creatives, members, speakers, mockups, logos, favicons, hero images, banner images, quizzes, testimonials, chat rooms, events, referrals) is tier-scoped (chapterId / countryId / global) with explicit inheritance (chapter-NULL → country-NULL → global).
- Every admin page has a **persistent scope switcher** in the top bar: Super Admin can navigate `Global ▸ Israel ▸ Tel Aviv` and see only that tier's data; Country Admin starts at `Israel ▸ *` and drills into chapters; Chapter Admin starts at `Tel Aviv` (no switcher — locked).
- Every content type has a tier-aware admin panel where any tier's admin (and Super Admin) can upload, edit, select, and relate the asset to each chapter.
- Country tier has its own brand-image columns (logo, favicon, hero, banner) so chapters can inherit from country, not just global.
- Email system fully per-tier: `fromName` / `fromEmail` / `replyTo` resolve chapter-override → country-default → global-default; `EmailStageTemplate` resolves chapter-override → country-default (NEW) → global-default; `ChapterEmailTemplateOverride` has admin UI.
- Per-chapter timezone propagates to every date formatter, RSVP, email scheduling, and iCal export.
- Per-chapter UI copy (chapter name in title template, header badge, email footer) replaces ~30 hard-coded `AI Salon Tel Aviv` strings.
- Public URLs stay flat for events (SEO preservation); chapter homepages live at `/c/[slug]` (already shipped); NEW city-root aliases `/[chapterSlug]` (e.g. `/tel-aviv`, `/montreal`) 301-redirect to `/c/[chapterSlug]` for marketing.

Migration philosophy — strictly additive, zero breaking changes:

- All schema changes are `ALTER TABLE ADD COLUMN` (nullable) + index. No drops. No renames. Existing V6/V7 code keeps running against the migrated DB.
- Backfill scripts are idempotent (only touch rows where the new column IS NULL).
- Feature-flagged rollout: every new scope filter is gated behind `process.env.V7_TIER_ENFORCEMENT !== "off"` so we can ship code, run the migration, then flip enforcement on a per-route basis.
- Every existing URL continues to resolve. New tier-prefixed URLs are added as aliases, not replacements.

Estimated scope:

Dimension	Count	Notes
Models touched (ALTER TABLE)	~20	QuizSession, Testimonial, ChatRoom, EventImage, PresentationFile, EventMockupDefault, SpeakerMessage, ConversationMessage, MemberTag, EventPrepQuestion, EventPrepSuggestion, EventAgendaItem, ChatMessage, ChatRoomMember, EmailEvent, TrackingLog, EmailFlowStep, QuizQuestion, QuizResponse, QuizParticipant, Country (new brand columns), ChapterEmailTemplateOverride (add countryId)
New models	2	`Creative` (unified marketing asset), `BrandAsset` (logo/favicon/hero/banner tier inheritance table — replaces SiteSetting/ChapterSetting ad-hoc approach)
New admin pages	6	`/admin/creatives`, `/admin/email/templates/overrides`, `/admin/countries/[id]/branding`, `/admin/global/branding`, `/admin/scope-switcher (component)`, `/admin/mockups/library (tier-scoped)`
Admin pages modified	14	All 16 existing pages get the scope switcher; `testimonials` permission bug fixed; `members` API scope bug fixed
API routes modified	~30	Every admin API route gets `scopeWhere()`; ~10 new routes for tier-scoped asset upload + Country brand CRUD + ChapterEmailTemplateOverride CRUD
Public routes modified	~10	Per-chapter timezone, per-chapter metadata, city-root aliases
Hard-coded strings replaced	~50 file:line refs	`Asia/Jerusalem` (15), `AI Salon Tel Aviv` (30+), `Tel Aviv Chapter` (5)
Migration phases	7	See Section 10

Top 5 risks:

1. **Scope-leak regression.** Adding `chapterId` to 20 more models means 20 more places to forget `scopeWhere()`. Mitigation: a `withScope()` middleware wrapper (Section 8) that REQUIRES every admin list endpoint to declare its scope; plus an automated scope-leak integration test that creates two chapters + data in each + logs in as each tier's admin + asserts zero cross-chapter rows in every API response.
2. **Email deliverability disruption.** Switching `fromEmail` resolution from env-var to per-chapter domain may break SPF/DKIM if chapters haven't configured DNS. Mitigation: feature-flag (`EMAIL_PER_TIER_ENABLED`); when off, falls back to current `ADMIN_EMAIL` env-var behavior; when on, the sender validates `Country.defaultEmailDomain` MX records exist before sending.
3. **Asset inheritance cache invalidation.** If chapter B inherits its logo from country Israel, and an admin uploads a new country-level logo, every chapter that "uses parent's logo" must re-resolve. Mitigation: `BrandAsset` table stores `(tier, tierId, kind, value, inheritFromParent)`; resolver is a single function `resolveBrandAsset(kind, chapterId)` that walks chapter → country → global at read time (no caching layer); if a CDN cache is added later, cache key includes `tierId` + `updatedAt` timestamp.
4. **Hard-coded `Asia/Jerusalem` → wrong times for Montreal.** Montreal users will see all event times in Israeli time until the timezone helper is wired everywhere. Mitigation: Phase 1 ships a single `getChapterTimezone(chapterId)` helper + replaces all 15 hard-coded call sites in one PR; integration test asserts every event-display route uses the chapter's timezone.
5. **Vercel Blob path migration.** Existing uploads live at `brand-assets/<file>`, `events/<eventId>/<file>`, etc. New convention `brand-assets/chapters/<chapterId>/<file>` would orphan old files. Mitigation: keep existing paths; new uploads use new convention; `BrandAsset` table maps `(tier, tierId, kind) → blobUrl` regardless of path; a one-off

`backfill-brand-asset-table.ts` script walks `SiteSetting` + `ChapterSetting` + `Chapter.heroImageUrl` and inserts rows into `BrandAsset` so all existing assets become tier-resolvable.

Section 1 — Data Model Completion

For every model in `prisma/schema.prisma`, here is the current state, target state, migration SQL, and inheritance rule. Models are grouped by V7-maturity.

1.1 Models that ALREADY have `chapterId` (V7-shipped) — verify scope, add country inheritance

These 13 models already have nullable `chapterId` + FK + index. **No schema change needed.** The work here is wiring `scopeWhere(scope)` into every query (some still don't — see Section 8) and writing backfill scripts for any new rows that get inserted without `chapterId` (e.g. new RSVPs created by public users).

Model	Current	Target	Migration
`User`	Has `countryId?`, `chapterId?`	Add `inheritFromParentBrand` Boolean? (whether member photos inherit chapter brand color) — OPTIONAL. Otherwise no change.	None.
`Event`	Has `chapterId?`, `isCrossChapter`. Still has legacy `chapter String @default("Tel Aviv")` cache column.	Keep legacy column for back-compat; mark deprecated; new code reads `chapterRef.name` instead.	None. Phase 7 cleanup task: drop the `chapter String` column (with a separate migration after all code stops reading it).
`Speaker`	Has `chapterId?` (denormalized from Event.chapterId at write time)	Ensure every `Speaker.create` writes `chapterId = event.chapterId` (audit `/api/admin/speakers` POST + `/api/admin/events/extract`).	None.
`EventRsvp`	Has `chapterId?`	Ensure RSVP creation writes `chapterId = event.chapterId` (audit `/api/events/[slug]/rsvp` + `/api/admin/rsvp` + bulk import).	None.
`EmailQueue`	Has `chapterId?`	Ensure flow worker writes `chapterId = flowStep.flow.chapterId ?? flowStep.template.chapterId ?? event.chapterId` (audit `flow-worker.ts` + `email-orchestrator/worker.ts`).	None.
`EmailRecipient`	Has `chapterId?` (denormalized from `EmailCampaign.chapterId`)	Ensure campaign send writes `chapterId = campaign.chapterId`.	None.
`EmailCampaign`	Has `chapterId?`	Already scoped via `emailModelWhere`. Add `countryId?` denormalized column for country-admin visibility? — NO, derive via `chapter.countryId`.	None.
`EmailTemplate`	Has `chapterId?`	Same.	None.

Model	Current	Target	Migration
`EmailStageTemplate`	Has `chapterId?`	Same.	None.
`EmailFlow`	Has `chapterId?`	Same.	None.
`EmailAudience`	Has `chapterId?`	Same.	None.
`ReferralVisit`	Has `chapterId?` (set by middleware based on `?utm_uid=` → referrer's chapter)	Verify middleware writes it.	None.
`ReferralAttribution`	Has `chapterId?`	Verify `record-conversion.ts` writes it.	None.

1.2 Models MISSING `chapterId` — the GAP list (need ALTER TABLE + backfill)

These models have NO tier scoping today. Each gets a nullable `chapterId` (and for some, also `countryId` denormalized) per the V7 pattern.

1.2.1 `QuizSession` + `QuizQuestion` + `QuizResponse` + `QuizParticipant`

- **Current.** `QuizSession` has `eventId?` but NO `chapterId`. Scoping today is via `eventId → Event.chapterId` (one join). Admin quiz list page (`/admin/quiz`) does NOT scope today (it lists all sessions globally).
- **Target.** Add `chapterId String?` to `QuizSession` (denormalized from `eventId → Event.chapterId` at session-creation time, or set explicitly by Super Admin for non-event quizzes). `QuizQuestion` / `QuizResponse` / `QuizParticipant` inherit scoping via `sessionId → QuizSession.chapterId` — NO chapterId column on these children (avoid data duplication).
- **Migration SQL.** ``sql ALTER TABLE "QuizSession" ADD COLUMN "chapterId" TEXT; CREATE INDEX "QuizSession_chapterId_idx" ON "QuizSession"("chapterId"); ALTER TABLE "QuizSession" ADD CONSTRAINT "QuizSession_chapterId_fkey" FOREIGN KEY ("chapterId") REFERENCES "Chapter"("id") ON DELETE SET NULL ON UPDATE CASCADE; -- Backfill: every session with an eventId gets that event's chapterId UPDATE "QuizSession" q SET "chapterId" = e."chapterId" FROM "Event" e WHERE q."eventId" = e."id" AND q."chapterId" IS NULL AND e."chapterId" IS NOT NULL; ``
- **Inheritance rule.** QuizSession is chapter-scoped (not inheritable). Country Admin sees all sessions in their country's chapters; Chapter Admin sees only their chapter's sessions.

1.2.2 `Testimonial` + `TestimonialLike`

- **Current.** `Testimonial` has NO `chapterId`. Chapter association is implicit via `eventId → Event.chapterId` (or `speakerId → Speaker.chapterId`). Admin moderation page (`/admin/testimonials`) lists ALL testimonials globally AND has the `me.role !== "ADMIN"` bug that excludes SUPER_ADMIN.
- **Target.** Add `chapterId String?` to `Testimonial`. `TestimonialLike` inherits via `testimonialId → Testimonial.chapterId` — NO chapterId on the like row. Add a country scope helper that resolves via `chapter.countryId`.

- **Migration SQL.** ``sql ALTER TABLE "Testimonial" ADD COLUMN "chapterId" TEXT; CREATE INDEX "Testimonial_chapterId_idx" ON "Testimonial"("chapterId"); ALTER TABLE "Testimonial" ADD CONSTRAINT "Testimonial_chapterId_fkey" FOREIGN KEY ("chapterId") REFERENCES "Chapter"("id") ON DELETE SET NULL ON UPDATE CASCADE; -- Backfill: chapter from event, then from speaker, then from speaker's user chapter UPDATE "Testimonial" t SET "chapterId" = e."chapterId" FROM "Event" e WHERE t."eventId" = e."id" AND t."chapterId" IS NULL AND e."chapterId" IS NOT NULL; UPDATE "Testimonial" t SET "chapterId" = s."chapterId" FROM "Speaker" s WHERE t."speakerId" = s."id" AND t."chapterId" IS NULL AND s."chapterId" IS NOT NULL; UPDATE "Testimonial" t SET "chapterId" = (SELECT u."chapterId" FROM "User" u WHERE u."id" = t."authorId") WHERE t."chapterId" IS NULL; ``
- **Inheritance rule.** Testimonial is chapter-scoped (not inheritable). Public `/testimonials?chapter=slug` shows community testimonials for that chapter + event/speaker/session testimonials where the parent is in that chapter.

1.2.3 `ChatRoom` + `ChatMessage` + `ChatRoomMember`

- **Current.** `ChatRoom` has `eventId?` (1:1 for event rooms). NO `chapterId`. GROUP rooms (no event) have no scoping at all.
- **Target.** Add `chapterId String?` to `ChatRoom`. `ChatMessage` / `ChatRoomMember` inherit via `roomId → ChatRoom.chapterId`. For event rooms, write `chapterId = event.chapterId` at room-creation time. For GROUP rooms, write the creator's `chapterId`.
- **Migration SQL.** ``sql ALTER TABLE "ChatRoom" ADD COLUMN "chapterId" TEXT; CREATE INDEX "ChatRoom_chapterId_idx" ON "ChatRoom"("chapterId"); ALTER TABLE "ChatRoom" ADD CONSTRAINT "ChatRoom_chapterId_fkey" FOREIGN KEY ("chapterId") REFERENCES "Chapter"("id") ON DELETE SET NULL ON UPDATE CASCADE; UPDATE "ChatRoom" r SET "chapterId" = e."chapterId" FROM "Event" e WHERE r."eventId" = e."id" AND r."chapterId" IS NULL AND e."chapterId" IS NOT NULL; UPDATE "ChatRoom" r SET "chapterId" = u."chapterId" FROM "User" u WHERE r."createdById" = u."id" AND r."chapterId" IS NULL AND u."chapterId" IS NOT NULL; ``
- **Inheritance rule.** ChatRoom is chapter-scoped (not inheritable). Country Admin can moderate any chat in their country's chapters. Chapter Admin can moderate only their chapter's chats.

1.2.4 `EventImage` + `PresentationFile`

- **Current.** Both have `eventId` but NO `chapterId`. Scoping today is via `eventId → Event.chapterId` (one join).
- **Target.** Add `chapterId String?` to both (denormalized from `eventId → Event.chapterId` at upload time). Lets the admin Images panel list images per-chapter without a join.
- **Migration SQL.** ``sql ALTER TABLE "EventImage" ADD COLUMN "chapterId" TEXT; CREATE INDEX "EventImage_chapterId_idx" ON "EventImage"("chapterId"); ALTER TABLE "EventImage" ADD CONSTRAINT "EventImage_chapterId_fkey" FOREIGN KEY ("chapterId") REFERENCES "Chapter"("id") ON DELETE SET NULL ON UPDATE CASCADE; UPDATE "EventImage" i SET "chapterId" = e."chapterId" FROM "Event" e WHERE i."eventId" = e."id" AND i."chapterId" IS NULL AND e."chapterId" IS NOT NULL; -- Same pattern for PresentationFile ALTER TABLE "PresentationFile" ADD COLUMN "chapterId" TEXT; CREATE INDEX "PresentationFile_chapterId_idx" ON "PresentationFile"("chapterId");

```
ALTER TABLE "PresentationFile" ADD CONSTRAINT "PresentationFile_chapterId_fkey"
FOREIGN KEY ("chapterId") REFERENCES "Chapter"("id") ON DELETE SET NULL ON UPDATE
CASCADE; UPDATE "PresentationFile" p SET "chapterId" = e."chapterId" FROM "Event"
e WHERE p."eventId" = e."id" AND p."chapterId" IS NULL AND e."chapterId" IS NOT
NULL; ``
```

- **Inheritance rule.** EventImage / PresentationFile are chapter-scoped (inherit via Event).

1.2.5 `EventMockupDefault`

- **Current.** `EventMockupDefault` has `eventId` + `imageUrl` (Vercel Blob PNG snapshot). NO `chapterId`.
- **Target.** Add `chapterId String?`. Lets chapter admin see their chapter's mockup history without joins.
- **Migration SQL.** Same pattern as EventImage.
- **Inheritance rule.** Chapter-scoped via Event.

1.2.6 `SpeakerMessage` + `ConversationMessage`

- **Current.** Both have NO `chapterId`. Speaker messages are scoped via `speakerId → Speaker.chapterId`. DMs are scoped via neither sender nor recipient chapter.
- **Target.** Add `chapterId String?` to both. SpeakerMessage writes `chapterId = speaker.chapterId` at creation. ConversationMessage writes `chapterId = sender.chapterId` (or recipient — they should match).
- **Migration SQL.** Same pattern.
- **Inheritance rule.** Chapter-scoped. Country Admin can audit messages in their country.

1.2.7 `MemberTag`

- **Current.** `MemberTag` has `userId` but NO `chapterId`. Tags are global today ("Speaker", "Builder", "Investor" — same labels across all chapters).
- **Target.** Add `chapterId String?` (NULL = global tag available to all chapters; non-NULL = chapter-specific tag). Lets a chapter define "Tel Aviv Investor" without polluting Montreal's tag list.
- **Migration SQL.** ``sql ALTER TABLE "MemberTag" ADD COLUMN "chapterId" TEXT; CREATE INDEX "MemberTag_chapterId_idx" ON "MemberTag"("chapterId"); ALTER TABLE "MemberTag" ADD CONSTRAINT "MemberTag_chapterId_fkey" FOREIGN KEY ("chapterId") REFERENCES "Chapter"("id") ON DELETE SET NULL ON UPDATE CASCADE; -- Existing tags stay NULL (global). No backfill needed. ``
- **Inheritance rule.** Tag list resolver returns: global tags (chapterId IS NULL) + chapter-specific tags (chapterId = scope.chapterId). Country Admin sees global + all chapters' tags in their country.

1.2.8 `EventPrepQuestion` + `EventPrepSuggestion`

- **Current.** Both have `eventId` but NO `chapterId`.
- **Target.** Add `chapterId String?` (denormalized from Event). Lets chapter admin see prep questions across their chapter's events without joins.
- **Migration SQL.** Same pattern.
- **Inheritance rule.** Chapter-scoped via Event.

1.2.9 `EmailEvent` + `TrackingLog`

- **Current.** Both have `campaignId` / `recipientId` / `queueId` but NO `chapterId`.
- **Target.** Add `chapterId String?` (denormalized from the parent EmailCampaign / EmailQueue at creation time). Lets analytics queries filter by chapter without joins.
- **Migration SQL.** Same pattern.
- **Inheritance rule.** Chapter-scoped via parent. Country Admin sees email analytics for their country; Chapter Admin sees only their chapter.

1.2.10 `EmailFlowStep`

- **Current.** `EmailFlowStep` has `flowId` but NO `chapterId`. Scoping is via `flowId → EmailFlow.chapterId`.
- **Target.** NO chapterId column — keep scoping via the parent flow (one join is fine; this is a small table). Document this as the explicit decision.
- **Migration SQL.** None.

1.3 NEW models to add

1.3.1 `BrandAsset` — unified tier-inheritable brand-image table

- **Problem.** Today, brand-image storage is fragmented: `SiteSetting[key=logoUrl/favicon/loginHero/loginBanner]` (global), `ChapterSetting[chapterId, key=...]` (per-chapter override), `Chapter.heroImageUrl` (per-chapter hero), `EmailStageTemplate.logoUrl` (per-template). There's NO country tier. The `chapter-brand-images.ts` resolver only handles 3 keys and only chapter → global fallback.
- **Target.** One unified table: `` ``` model BrandAsset { id String @id @default(cuid()) tier String // "global" | "country" | "chapter" countryId String? chapterId String? kind String // "logo" | "favicon" | "heroLogin" | "heroChapter" | "banner" | "ogImage" value String // Vercel Blob URL or absolute URL inheritFromParent Boolean @default(false) // when true, ignore value and walk up the tier tree updatedBy String? updatedAt DateTime @updatedAt @@unique([tier, countryId, chapterId, kind]) @@index([chapterId]) @@index([countryId]) } ``` ``
- **Semantics.**
  - `tier="global"`: row applies to all chapters. `countryId` and `chapterId` are NULL.
  - `tier="country"`: row applies to all chapters in that country. `chapterId` is NULL.
  - `tier="chapter"`: row applies to that specific chapter.
  - `inheritFromParent=true`: the chapter/country explicitly opts out of having its own asset; resolver walks up.
- **Resolver.** `` ```ts async function resolveBrandAsset(kind: BrandAssetKind, chapterId?: string, countryId?: string): Promise<string> { // 1. Try chapter override if (chapterId) { const r = await db.brandAsset.findUnique({ where: { tier_countryId_chapterId_kind: { tier: "chapter", countryId: null, chapterId, kind } } }); if (r && !r.inheritFromParent && r.value) return r.value; } // 2. Try country override if (countryId) { const r = await db.brandAsset.findUnique({ where: { tier_countryId_chapterId_kind: { tier: "country", countryId, chapterId: null, kind } } }); if (r && !r.inheritFromParent && r.value) return r.value; } //

3. Fall back to global const g = await db.brandAsset.findUnique({ where: { tier_countryId_chapterId_kind: { tier: "global", countryId: null, chapterId: null, kind } } }); if (g && g.value) return g.value; // 4. Final fallback: hard-coded default return DEFAULTS[kind]; } ``

- **Migration.** `CREATE TABLE "BrandAsset" (...); CREATE UNIQUE INDEX ...; CREATE INDEX ...;`. Plus a backfill script `scripts/backfill-brand-asset-table.ts` that reads existing `SiteSetting` + `ChapterSetting` + `Chapter.heroImageUrl` + `EmailStageTemplate.logoUrl` and inserts equivalent rows into `BrandAsset`.
- **Coexistence.** `SiteSetting` + `ChapterSetting` continue to exist (back-compat). The new `BrandAsset` table is the source of truth going forward; `chapter-brand-images.ts` is rewritten to read from `BrandAsset` first, then fall back to the old tables.

1.3.2 `Creative` — unified marketing-asset table (mockups + banners + ads + social posts)

- **Problem.** No dedicated marketing-creative model exists. Today creatives are spread across `EventMockupDefault` (per-event mockup PNGs), `EventImage` (event photos), `EmailStageTemplate.logoUrl` (email logos), and ad-hoc Vercel Blob uploads.
- **Target.** New table: `` model Creative { id String @id @default(cuid()) name String // "TLV July 2024 hero banner" kind String // "mockup" | "banner" | "socialPost" | "adAsset" | "emailHeader" | "linkedinCard" tier String // "global" | "country" | "chapter" countryId String? chapterId String? blobUrl String // Vercel Blob URL thumbnailUrl String? // optional smaller preview width Int? height Int? mimeType String @default("image/png") tagsJson String @default("[]") // ["hero", "tlv", "july-2024"] eventId String? // optional link to a specific event uploaderId String createdAt DateTime @default(now()) updatedAt DateTime @updatedAt @@index([tier, countryId, chapterId]) @@index([kind]) @@index([chapterId]) } ``
- **Migration.** `CREATE TABLE "Creative" (...);`. Backfill: copy every `EventMockupDefault.imageUrl` into `Creative(kind="mockup", chapterId=event.chapterId, eventId=event.id)`; copy every `EventImage.fileUrl` into `Creative(kind="eventPhoto", ...)`.
- **Admin UI.** New `/admin/creatives` page (Section 4).

1.4 `Country` — add brand-image columns + inheritable defaults

- **Current.** `Country` has `defaultEmailDomain`, `defaultFromName`, `defaultReplyTo`, `flagEmoji`. NO brand-image columns. NO `defaultTimezone`.
- **Target.** Add columns: `` defaultLogoUrl String? // country-level logo (chapters without their own inherit this) defaultFaviconUrl String? defaultHeroUrl String? // chapter-landing hero defaultBannerUrl String? // OG image / login banner defaultTimezone String? // "Asia/Jerusalem" for Israel, "America/Montreal" for Canada defaultLocale String? // "en-US" / "he-IL" / "fr-CA" ``
- **Migration SQL.** ``sql ALTER TABLE "Country" ADD COLUMN "defaultLogoUrl" TEXT, ADD COLUMN "defaultFaviconUrl" TEXT, ADD COLUMN "defaultHeroUrl" TEXT, ADD COLUMN "defaultBannerUrl" TEXT, ADD COLUMN "defaultTimezone" TEXT, ADD COLUMN "defaultLocale" TEXT; -- Backfill: Israel gets Asia/Jerusalem + en-US UPDATE "Country" SET "defaultTimezone" = 'Asia/Jerusalem', "defaultLocale" = 'en-US' WHERE "code" = 'IL'; ``

- **Inheritance rule.** Chapter timezone resolver: `chapter.timezone ?? country.defaultTimezone ?? 'UTC'`. Chapter locale resolver: `chapter.locale ?? country.defaultLocale ?? 'en-US'` (add `Chapter.locale String?` column too).

1.5 `ChapterEmailTemplateOverride` — add `countryId` for country-tier overrides

- **Current.** `ChapterEmailTemplateOverride` has `chapterId` + `stageTemplateId`. NO `countryId`. So today overrides are chapter-only.
- **Target.** Two options:
  - **Option A (recommended).** Keep the table chapter-only. For country-level overrides, use `EmailStageTemplate.chapterId` with a "country chapter" sentinel? — NO, confusing.
  - **Option B (recommended).** Rename concept to `TierEmailTemplateOverride` and add `tier` + `countryId` columns.
- **Decision.** Option B. New schema: `` ``` model TierEmailTemplateOverride { id String @id @default(cuid()) tier String // "country" | "chapter" countryId String? // set when tier="country" chapterId String? // set when tier="chapter" stageTemplateId String logoUrl String? subject String? htmlBody String? fromName String? // NEW — per-tier from-name override fromEmail String? // NEW — per-tier from-email override replyTo String? // NEW — per-tier reply-to override isActive Boolean @default(true) updatedAt DateTime @updatedAt @@unique([tier, countryId, chapterId, stageTemplateId]) @@index([chapterId]) @@index([countryId]) } ``` ``
- **Migration.** `ALTER TABLE "ChapterEmailTemplateOverride" RENAME TO "TierEmailTemplateOverride";` + add columns. Backfill: existing rows get `tier='chapter'`, `countryId = (SELECT countryId FROM Chapter WHERE id = chapterId)`.
- **Resolver.** `` ```ts async function resolveEmailTemplate(stageTemplateId: string, chapterId?: string, countryId?: string) { // 1. Chapter override if (chapterId) { ... } // 2. Country override if (countryId) { ... } // 3. EmailStageTemplate chapterId (per-chapter template) // 4. EmailStageTemplate global (chapterId IS NULL) } ``` ``

1.6 Inheritance resolution table — summary

For each content type, here is the fallback order:

Content type	Tier-1 (chapter)	Tier-2 (country)	Tier-3 (global)	NULL semantics
Logo	`BrandAsset[tier=chapter, kind=logo]`	`BrandAsset[tier=country, kind=logo]`	`BrandAsset[tier=global, kind=logo]`	`inheritFromParent=true` means walk up; NULL `value` (with `inherit=false`) means "no logo"
Favicon	same	same	same	same
Hero (chapter landing)	`Chapter.heroImageUrl` (keep) OR `BrandAsset[tier=chapter, kind=heroChapter]`	`BrandAsset[tier=country, kind=heroChapter]`	`BrandAsset[tier=global, kind=heroChapter]`	NULL = render gradient-only hero

Content type	Tier-1 (chapter)	Tier-2 (country)	Tier-3 (global)	NULL semantics
Hero (login)	`BrandAsset[tier=chapter, kind=heroLogin]`	`BrandAsset[tier=country, kind=heroLogin]`	`BrandAsset[tier=global, kind=heroLogin]`	same
Banner (OG / login)	`BrandAsset[tier=chapter, kind=banner]`	`BrandAsset[tier=country, kind=banner]`	`BrandAsset[tier=global, kind=banner]`	same
Email from-name	`TierEmailTemplateOverride[tier=chapter].fromName`	`TierEmailTemplateOverride[tier=country].fromName`	`Country.defaultFromName`	`ADMIN_EMAIL` env-var name
Email from-email	`TierEmailTemplateOverride[tier=chapter].fromEmail`	`TierEmailTemplateOverride[tier=country].fromEmail`	`Country.defaultEmailDomain` (concat with `noreply@`)	`ADMIN_EMAIL` env-var
Email reply-to	`TierEmailTemplateOverride[tier=chapter].replyTo`	`TierEmailTemplateOverride[tier=country].replyTo`	`Country.defaultReplyTo`	`ADMIN_EMAIL` env-var
Email stage template body	`TierEmailTemplateOverride[tier=chapter].htmlBody`	`TierEmailTemplateOverride[tier=country].htmlBody`	`EmailStageTemplate` (global default; `chapterId IS NULL`)	`N/A`
Email stage template subject	same path, `.subject`	same	same	`N/A`
Email stage template logo	`TierEmailTemplateOverride[tier=chapter].logoUrl`	`TierEmailTemplateOverride[tier=country].logoUrl`	`EmailStageTemplate.logoUrl`	`EMAIL_BRAND_LOGO_URL` env-var
Timezone	`Chapter.timezone`	`Country.defaultTimezone`	`UTC`	NULL = use UTC
UI display name	`Chapter.name`	`Country.name`	"AI Salon" (no city)	`N/A`
WhatsApp group URL	`Chapter.whatsappGroupUrl`	`SiteSetting[whatsappGroupUrl]` (global)	—	NULL = hide WhatsApp button
LinkedIn URL	`Chapter.linkedinUrl`	`SiteSetting[linkedinUrl]`	—	NULL = hide LinkedIn button
Quiz / Testimonial / ChatRoom / EventImage	chapter-scoped via `chapterId`	country admin sees all in country	super admin sees all	NULL = country-admin sees it (legacy rows before backfill); chapter-admin doesn't
Email template (reusable)	`EmailTemplate[chapterId=X]`	(no country tier for reusable templates — only stage templates get country overrides)	`EmailTemplate[chapterId IS NULL]`	NULL = global template, available to all chapters

Content type	Tier-1 (chapter)	Tier-2 (country)	Tier-3 (global)	NULL semantics
Email flow	`EmailFlow[chapterId=X]`	(no country tier — flows are chapter-only)	`EmailFlow[chapterId IS NULL]`	NULL = global flow
Email audience	`EmailAudience[chapterId=X]`	(no country tier)	`EmailAudience[chapterId IS NULL]`	NULL = global audience
Event	`Event[chapterId=X]` (+ cross-chapter events visible to all chapters in country)	`Event[chapterRef.countryId=Y]`	All events	NULL = unscoped (legacy; should be backfilled)
Member (User)	`User[chapterId=X]` OR `User[countryId=Y, chapterId IS NULL]` (country member without chapter)	`User[countryId=Y]`	All users	NULL = unscoped (legacy)
Speaker	`Speaker[chapterId=X]`	`Speaker[chapter.countryId=Y]`	All speakers	NULL = unscoped
Referral visit / attribution	`ReferralVisit[chapterId=X]`	`ReferralVisit[chapter.countryId=Y]`	`All`	NULL = unscoped

NULL semantics rule. For INHERITABLE assets (logos, favicons, brand images, email defaults), NULL means "inherit from parent tier". For NON-INHERITABLE entities (events, members, speakers), NULL means "unscoped — backfill to the chapter of the row that owns it" (or, for legacy rows, to Israel/Tel Aviv via the seed script).

Section 2 — Permission & Scope Model

2.1 Role hierarchy (final, no changes from V7)

Role	Rank	Scope	Source of truth
`SUPER_ADMIN`	4	Global (all countries, all chapters)	`SUPER_ADMIN_EMAILS` hard-coded Set in `permissions.ts`. Currently `{"eze@massapro.com"}`. To add another Super Admin, edit code + redeploy. (Open question: should this become DB-driven? See Section 12.)
`ADMIN`	3	Country (one country + all chapters in it)	`User.countryId`. Set by Super Admin via EditMemberDialog.
`CHAPTER_ORGANIZER`	2	Chapter (one chapter only)	`User.chapterId` + `User.countryId` (both required). Replaces V6 `CO_HOST`.
`CO_HOST`	2 (legacy)	Per-event via `EventCoHost` table	Same rank as `CHAPTER_ORGANIZER`. V6 pattern retained.
`MEMBER`	1	`Default`	Auto-set on signup. `User.countryId` set if signed up via `/c/[chapterSlug]`; `User.chapterId` set on first RSVP (V7 README Q5 — TODO, currently NULL until backfilled).
`SPEAKER`	0 (legacy, outside inheritance)	`N/A`	Only gets `eventprep.view`. V7 README says migrate to `MEMBER`.

No role changes needed. The hierarchy is correct. The work is in CRUD matrix enforcement + scope switcher UX.

2.2 CRUD matrix per content type per role

(Y = yes; N = no; S = self-only; C = own chapter only; O = own country only; G = global)

Content type	Action	SUPER_ADMIN	ADMIN (country)	CHAPTER_ORG (chapter)	MEMBER
Countries	view	Y	Y (own)	N	N
	create / edit / delete	Y	N	N	N
Chapters	view	Y	Y (own country)	Y (own)	N
	create / edit	Y	Y (own country)	N	N
	delete	Y	N	N	N
Members	view	Y (all)	Y (own country)	Y (own chapter)	N
	edit	Y	Y (own country)	Y (own chapter)	S (self)
	change role	Y	Y (CHAPTER_ORG + MEMBER, own country)	N	N
	delete	Y	N	N	N
	bulk import	Y	Y (own country)	Y (own chapter)	N
Events	view	Y	Y (own country)	Y (own chapter)	Y (signed-in)
	create	Y	Y (own country)	Y (own chapter)	N
	edit	Y	Y (own country)	Y (own chapter)	N
	delete	Y	N	N	N
	set `isCrossChapter`	Y	N	N	N
Speakers	view	Y	Y	Y	N
	create / edit	Y	Y	Y	N
	delete	Y	N	N	N
Registrants / RSVPs	view	Y	Y	Y	S (own RSVPs)
	edit	Y	Y	Y	N
	bulk import	Y	Y	Y	N
Emails — campaigns	view	Y	Y (own country + global)	Y (own chapter + global)	N
	create / send	Y	Y (own country)	Y (own chapter)	N
	delete	Y	N	N	N
Emails — templates	view	Y	Y (own country + global)	Y (own chapter + global)	N
	create / edit	Y	Y (own country)	Y (own chapter)	N

Content type	Action	SUPER_ADMIN	ADMIN (country)	CHAPTER_ORG (chapter)	MEMBER
	delete	Y	N	N	N
Emails — stage templates (5-stage)	view	Y	Y	Y	N
	create / edit	Y	Y (own country)	Y (own chapter)	N
	delete (non-default)	Y	N	N	N
Emails — `TierEmailTemplateOverride`	view	Y	Y (own country)	Y (own chapter)	N
	create / edit	Y	Y (own country)	Y (own chapter)	N
	delete	Y	Y (own country)	Y (own chapter)	N
Emails — flows / audiences	view	Y	Y (own country + global)	Y (own chapter + global)	N
	create / edit	Y	Y (own country)	Y (own chapter)	N
	delete	Y	N	N	N
Creatives (mockups, banners, ads)	view	Y	Y (own country)	Y (own chapter)	N
	upload / edit	Y	Y (own country)	Y (own chapter)	N
	delete	Y	Y (own country)	Y (own chapter)	N
Logos / favicons / heroes / banners (`BrandAsset`)	view	Y	Y (own country + global)	Y (own chapter + global)	N
	upload custom (chapter tier)	Y	Y (own country)	Y (own chapter)	N
	upload custom (country tier)	Y	Y (own country)	N	N
	upload custom (global tier)	Y	N	N	N
	toggle `inheritFromParent`	Y	Y (own country)	Y (own chapter)	N
Quizzes	view	Y	Y	Y	N (only sessions they joined)
	create / host	Y	Y	Y	N
	delete	Y	N	N	N
Testimonials	view (public feed)	Y	Y	Y	Y
	moderate (hide/feature)	Y	Y (own country)	Y (own chapter)	N
	delete	Y	Y (own country)	Y (own chapter)	S (own)

Content type	Action	SUPER_ADMIN	ADMIN (country)	CHAPTER_ORG (chapter)	MEMBER
Chat rooms	view	Y	Y	Y	Y (rooms they're in)
	moderate (delete msg, kick)	Y	Y (own country)	Y (own chapter)	N
Reports / analytics	view	Y (all)	Y (own country)	Y (own chapter)	N
Brand settings (global)	view / edit	Y	N	N	N

2.3 `getUserScope()` / `scopeWhere()` extensions

The current `getUserScope()` in `src/lib/permissions.ts` already returns `{kind:"global"|"country"|"chapter"|"none"}`. **No change to the type signature.**

New helpers needed:

1. `getEffectiveScope(userId, override?): Promise<UserScope>` — same as `getUserScope()` BUT honors a "scope override" stored in the user's session or a cookie (for the scope switcher UI, see 2.4). The override can only NARROW the user's natural scope (Super Admin can override to "country:IL" or "chapter:tel-aviv"; Country Admin can override to "chapter:tel-aviv"; Chapter Admin cannot override — they're locked).
2. `scopeBrandAssetWhere(scope): Prisma.Where` — for `BrandAsset` queries (uses `tier` + `countryId` + `chapterId`).
3. `scopeCreativeWhere(scope): Prisma.Where` — for `Creative` queries.
4. `scopeTestimonialWhere(scope): Prisma.Where` — for `Testimonial` queries (uses `chapter.countryId` for country scope).
5. `scopeQuizWhere(scope): Prisma.Where` — for `QuizSession` queries.
6. `scopeChatWhere(scope): Prisma.Where` — for `ChatRoom` queries.
7. `scopeImageWhere(scope): Prisma.Where` — for `EventImage` / `PresentationFile` queries (uses `chapter.countryId` for country scope).
8. `scopeMockupWhere(scope): Prisma.Where` — for `EventMockupDefault` queries.
9. `scopeTierEmailTemplateOverrideWhere(scope): Prisma.Where` — for the renamed override table.

All of these follow the same pattern as `scopeChapterWhere(scope)` — global returns `{}`, country returns `{ chapter: { countryId: scope.countryId } }`, chapter returns `{ chapterId: scope.chapterId }`, none returns `{ id: "___NEVER___" }`.

2.4 Scope switcher UX

Problem. Today every admin page renders a read-only "scope badge" reflecting the user's natural scope. A Super Admin sees "Global scope" on every page — they cannot drill into a specific country or chapter from the UI; they have to use the per-page `<CountryChapterScopeFilter>` client-side row filter, which is purely cosmetic (doesn't change server queries).

Target. Add a persistent scope switcher in `AppHeader` (top-right, next to the user menu). It's a dropdown showing the current effective scope as a breadcrumb:

```
• Global  ▼
──────────────────
• Global                    (reset)
──────────────────
• Israel
  └ • Tel Aviv
  └ • Jerusalem (NEW)
• Canada
  └ • Montreal
──────────────────
+ Add country/chapter (Super Admin only)
```

Behavior:

- **SUPER_ADMIN.** Sees the full tree. Clicking any leaf sets the effective scope (stored in a `scope` cookie + the user's session JWT). All admin pages re-query using the narrowed scope. A "• Global" item at the top resets to global.
- **ADMIN.** Sees only their country + its chapters (no other countries, no "Global" reset). Default effective scope = country.
- **CHAPTER_ORGANIZER.** No switcher rendered. Effective scope = their chapter, locked.
- **MEMBER.** No admin header at all (no `/admin` access).

The switcher persists across page navigations (cookie-based). The server reads the override on every request via `getEffectiveScope(userId, req.cookies.scope)`.

Implementation.

- New component `<ScopeSwitcher />` in `src/components/ais/scope-switcher.tsx`.
- New API route `POST /api/admin/scope` — body `{tier: "global"|"country"|"chapter", countryId?, chapterId?}`. Validates the override is narrower than the user's natural scope (a Country Admin cannot override to "global"; a Chapter Admin cannot override at all). Sets the `scope` cookie (HttpOnly, SameSite=Lax, 30-day expiry) + returns the new scope.
- Update `getCurrentUser()` in `auth-guards.ts` to read the scope cookie and pass it to `getEffectiveScope()`.

2.5 Login flow per role

- **SUPER_ADMIN.** Logs in → redirected to `/admin`. Scope switcher shows "• Global". They can navigate to any country/chapter via the switcher.
- **ADMIN.** Logs in → redirected to `/admin`. Scope switcher shows "• Israel ▼" with their country pre-selected. They can drill into chapters within Israel only.
- **CHAPTER_ORGANIZER.** Logs in → redirected to `/admin`. No switcher. Every admin page is locked to their chapter.
- **MEMBER.** Logs in → redirected to `/events`. No admin access.

2.6 Inventory bugs to fix

1. `/admin/testimonials` **role gate bug.** Today: `if (me.role !== "ADMIN") redirect("/events")`. This excludes SUPER_ADMIN (because SUPER_ADMIN !== "ADMIN"). Fix: `if (!can(me.role, "members.view") && me.role !== ROLES.SUPER_ADMIN) redirect("/events")` OR better, use `can(me.role, "testimonials.moderate")` (new permission, granted to ADMIN+, CHAPTER_ORGANIZER for own chapter). Plus add `scopeTestimonialWhere(scope)` to the page query.
2. `/api/admin/members` **GET doesn't scope.** Today: returns ALL members. Fix: `const scope = await getUserScope(me.id); const where = { archivedAt: null, ...scopeUserWhere(scope) }; const members = await db.user.findMany({ where, ... });`. Also accept `?countryId=X&chapterId=Y` query params for the client-side filter, validated against the caller's scope.
3. **Duplicate scope helpers in** `src/lib/v7-scope.ts`**.** Delete the file (it's dead code). The production versions in `permissions.ts` are correct. Update any imports.
4. `requireAdmin()` **in** `src/lib/admin-auth.ts`**.** Legacy, hard-codes `role !== "ADMIN"` (no SUPER_ADMIN, no scope). Used only by older routes. Audit which routes still use it and migrate them to `getCurrentUser()` + `can()`. Then delete `admin-auth.ts`.

Section 3 — URL Routing & Public Site

3.1 Decision: Option C (Hybrid)

- **Public events:** stay flat — `/events/[slug]` and `/e/[slug]` (preserve SEO, preserve existing shares + emails).
- **Chapter homepages:** live at `/c/[chapterSlug]` (already shipped) — e.g. `/c/tel-aviv`, `/c/montreal`.
- **City-root aliases:** NEW short URLs `/<chapterSlug>` (e.g. `/tel-aviv`, `/montreal`) that 301-redirect to `/c/<chapterSlug>`. Marketing-friendly; no SEO cost (redirect).
- **Country homepages:** NEW `/<countrySlug>` (e.g. `/israel`, `/canada`) — shows a country landing page listing all chapters in that country. (If empty, redirect to the country's first chapter.)

Justification. Option A (`/[country]/[chapter]/events/[slug]`) breaks every existing event URL — bad for SEO, bad for shared email links. Option B (flat) misses the marketing win of `tel-aviv.aisalon.org` or `aisalon.massapro.com/tel-aviv`. Option C gives marketing-friendly chapter URLs without breaking event SEO.

3.2 Next.js route segments to add/modify

Add:

- `src/app/[chapterSlug]/page.tsx` — city-root alias. Reads `params.chapterSlug`, looks up `Chapter` by slug, returns `redirect(\/c/${chapterSlug}\, { type: "permanent" })` (301). If no chapter matches, returns `notFound()`.
- `src/app/[countrySlug]/page.tsx` — country landing page. Looks up `Country` by slug, lists its chapters, links to each `/c/[chapterSlug]`. If country has 1 chapter, redirects there.
- `src/app/[countrySlug]/[chapterSlug]/page.tsx` — explicit two-tier chapter URL (e.g. `/israel/tel-aviv`) — also redirects to `/c/[chapterSlug]` for canonical simplicity. (Optional, for marketing flexibility.)

Modify:

- `src/app/c/[chapterSlug]/page.tsx` — already exists; keep as canonical chapter URL. Add `generateMetadata` per-chapter title + favicon + OG image (already wired). Add per-chapter timezone-aware event listings.
- `src/app/events/page.tsx` — read `?chapter=slug` query param (already auto-selects chapter filter). NEW: also resolve chapter branding (favicon, OG image) from the `?chapter=` param so the events page itself reflects the chapter context.
- `src/app/e/[slug]/page.tsx` — public event page. NEW: resolve chapter from `event.chapterId`, apply chapter branding (favicon, OG image, header badge, page title `"<Event title> — AI Salon <Chapter.name>"`), use `chapter.timezone` for all date displays, use `chapter.name` in the footer instead of hard-coded "Tel Aviv Chapter".
- `src/app/layout.tsx` — replace hard-coded `%s — AI Salon Tel Aviv` title template with a chapter-aware template via `generateMetadata()` reading the route's chapter context. For routes without chapter context (e.g. `/`), use `AI Salon` (global).
- `src/app/page.tsx` — currently a redirect. NEW: render a global landing page (hero with chapter picker, "Find your chapter" CTA, list of countries → chapters). Auto-redirect to nearest chapter if `?chapter=slug` or cookie set; otherwise show the picker.

3.3 Redirect strategy

- `/[chapterSlug]` → `/c/[chapterSlug]` (301 permanent). Match against the `Chapter.slug` column. If no match, fall through to Next.js 404.
- `/[countrySlug]` → country landing page (no redirect; renders HTML).
- Existing `/events/[slug]` and `/e/[slug]` URLs — NO redirect (preserve as-is).
- Existing `/c/[chapterSlug]` URLs — NO redirect (canonical).
- NEW chapter-prefixed URLs (if added later): `/c/[chapterSlug]/events/[slug]` — would require a redirect from `/events/[slug]`. DEFERRED (not in this plan).

3.4 Chapter branding resolution per route

Every server-rendered page resolves branding via:

```
async function getChapterBrandingForRoute(req): Promise<{ chapter?: Chapter; country?: Count
ry; branding: PublicBranding }> {
  // 1. Try URL: /c/[chapterSlug] or /[chapterSlug]
  // 2. Try ?chapterSlug= query param
  // 3. Try ?chapter= query param (alias)
  // 4. Try scope cookie (if user is signed-in admin with narrowed scope)
  // 5. Try user's User.chapterId (if signed-in)
  // 6. Default: global branding (no chapter)
}
```

Returns the resolved `chapterId` (or null) + `countryId` (or null) + the resolved `BrandAsset` values (logo, favicon, hero, banner, OG image). The page's `generateMetadata()` uses these for `<title>`, `<link rel="icon">`, `<meta property="og:image">`.

3.5 Homepage behavior

- **Anonymous visitor** at `/` — sees a global landing page:
  - Hero: "Find your AI Salon chapter"
  - Country picker (large cards with flag + name) → on click, drills into chapter list
  - "Upcoming events across all chapters" (mixed feed, chapter-badged)
  - "Don't see your city? Start a chapter" CTA (mailto:)
- **Signed-in member** at `/` — redirects to `/events` (current behavior, preserved).
- **Signed-in admin** at `/` — redirects to `/admin` (current behavior, preserved).
- Auto-redirect: if `?chapter=slug` is in the URL, redirect to `/c/[slug]` immediately. (Cookie-based "remember my chapter" is a future enhancement; not in this plan.)

Section 4 — Admin UI Completion

For every content type, here is the admin panel design. All panels share:

- Top bar with `<ScopeSwitcher />` (Section 2.4)
- Page-level scope badge (existing — preserved)
- List view: table or grid with chapter/country column + tier filter
- Edit form: modal or side-drawer with tier-aware fields
- Permission gates: `can(me.role, "<perm>")` server-side + client-side button visibility

4.1 Email template manager — `/admin/email` (existing, extend)

Existing. 3 sub-tabs: campaigns, orchestrator (queue), flows. Tier-scoped via `emailModelWhere`. Missing: `TierEmailTemplateOverride` UI.

Add:

- New sub-tab "Stage template overrides" at `/admin/email/templates/overrides`. Lists all 5 stage templates (Awareness, Reminder, Final Prep, Day-Of, Recap) with a 3-column matrix:
  - Column 1: Global template (read-only except Super Admin)
  - Column 2: Country override (editable by Admin+ for own country; "Use parent" toggle)
  - Column 3: Chapter override (editable by Chapter Organizer+ for own chapter; "Use parent" toggle)
- Click a cell → opens editor with fields: `logoUrl` (upload or paste), `subject`, `htmlBody` (rich text via `@mdxeditor/editor`), `fromName`, `fromEmail`, `replyTo`, `isActive` toggle, "Use parent's value" toggle per field (so a chapter can override the body but inherit the subject).
- New API routes:
  - `GET /api/admin/email/templates/overrides?tier=country&id=IL` — list overrides for a tier
  - `POST /api/admin/email/templates/overrides` — create/update an override
  - `DELETE /api/admin/email/templates/overrides/[id]` — delete an override (revert to parent)
- Permission gate: ADMIN+ for own country; CHAPTER_ORGANIZER for own chapter; SUPER_ADMIN for global edits.

4.2 Creative asset manager — `/admin/creatives` (NEW)

Unified Media Library vs. **separate panels** — decision: **unified**. Reason: the user listed "mockups, logos, favicons, hero images, banner images" together; a single Media Library with strong filtering is more usable than 5 separate panels. Per-chapter override UI lives in the same library (select an asset, "assign to chapter X as their logo").

Page layout:

- Top: filter bar — `Kind` (mockup/banner/socialPost/adAsset/emailHeader/linkedinCard), `Tier` (global/country/chapter), `Country` (dropdown), `Chapter` (dropdown), `Tags` (multi-select), `Search` (name)
- Main: grid of asset cards (thumbnail, name, kind badge, tier badge, chapter/country flag, uploader, date)
- Right side: detail drawer — preview, metadata, "Assign as..." dropdown (logo for chapter X / favicon for country Y / hero for chapter Z), delete button
- Upload: drag-drop zone at top; modal asks for `name`, `kind`, `tier` (defaults to caller's natural scope), `countryId/chapterId` (locked to caller's scope), `tags`

Permission gates:

- View: ADMIN+ for own country; CHAPTER_ORGANIZER for own chapter; SUPER_ADMIN for all.
- Upload: same as view + the asset's tier must be within the caller's scope (Chapter Organizer cannot upload to country tier; Admin cannot upload to global tier).
- Assign as brand asset: ADMIN+ for own country's chapters; CHAPTER_ORGANIZER for own chapter; SUPER_ADMIN for any.
- Delete: own assets only (uploader) OR SUPER_ADMIN.

API routes:

- `GET /api/admin/creatives?tier=X&countryId=Y&chapterId=Z&kind=K` — list (scoped)
- `POST /api/admin/creatives` — upload (multipart to Vercel Blob + create row)
- `PATCH /api/admin/creatives/[id]` — edit metadata
- `DELETE /api/admin/creatives/[id]` — delete (also deletes Vercel Blob)
- `POST /api/admin/creatives/[id]/assign` — body `{kind: "logo"|"favicon"|..., tier, countryId?, chapterId?}` — creates/updates a `BrandAsset` row pointing at this creative's `blobUrl`

4.3 Brand asset manager — `/admin/branding` (NEW; replaces `/admin/images`)

Problem. `/admin/images` today is a hybrid: it lists Vercel Blob `brand-assets/` files + lets Super Admin "select" one as global favicon/loginHero/loginBanner + lets admins "select" per-chapter overrides. It conflates asset storage with brand-asset assignment.

Target. Split:

- `/admin/creatives` — asset storage (Section 4.2).
- `/admin/branding` — brand-asset assignment. Three sub-pages: `/admin/branding/global` (Super Admin only), `/admin/branding/countries/[id]` (Admin for own country), `/admin/branding/chapters/[id]` (Chapter Organizer+ for own chapter).

`/admin/branding/global` **layout:**

- 6 cards: Logo, Favicon, Login Hero, Chapter Hero (default), Banner (OG/Login), Email Header
- Each card: preview thumbnail, "Upload custom" button (uploads to `BrandAsset[tier=global]`), "Clear" button (sets `inheritFromParent=true`, but at global tier there's no parent — so really "reset to DEFAULTS")
- Below: "Email defaults" form — `defaultFromName`, `defaultEmailDomain`, `defaultReplyTo` (these go on a new `GlobalSetting` table OR stay as `SiteSetting` keys; recommend `SiteSetting` for back-compat).

`/admin/branding/countries/[id]` **layout:**

- Same 6 cards. Each card: preview of the currently-effective asset (resolved via `resolveBrandAsset(kind, null, countryId)`), "Upload custom" button (uploads to `BrandAsset[tier=country, countryId=id]`), "Use global" toggle (sets `inheritFromParent=true`).
- Below: country-level email defaults form (overrides `Country.defaultFromName` etc.) + country-level timezone + locale.

`/admin/branding/chapters/[id]` **layout:**

- Same 6 cards. Each card: preview of currently-effective asset (resolved via `resolveBrandAsset(kind, chapterId, countryId)`), "Upload custom" button, "Use country's" toggle (sets `inheritFromParent=true`, walks up to country), "Use global's" toggle (force global).
- Below: chapter settings form (existing `chapter-editor.tsx` fields — name, slug, city, timezone, WhatsApp, LinkedIn, hero image).

API routes:

- `GET /api/admin/branding?tier=X&countryId=Y&chapterId=Z` — list all `BrandAsset` rows for a tier
- `POST /api/admin/branding` — body `{kind, tier, countryId?, chapterId?, value, inheritFromParent}` — create/update
- `DELETE /api/admin/branding/[id]` — delete (revert to parent)
- `POST /api/admin/branding/upload` — multipart upload to Vercel Blob + return URL (then caller POSTs to `/api/admin/branding` with the URL)

4.4 Member manager — `/admin` (existing, extend)

Existing. Members table with search, tag assignment, EditMemberDialog with V7 hierarchy assignment. Page-level scoped. API NOT scoped (bug).

Fix + extend:

- Fix `/api/admin/members` GET to scope via `scopeUserWhere(scope)`.
- Add tier filter chips at top: "All" / "Israel" / "Tel Aviv" / "Montreal" (depends on caller's scope) — these are URL query params that re-query the server (not just client-side filter).
- Add "Country · Chapter" column (already exists).
- Add bulk actions: bulk-assign-scope (exists), bulk-tag (exists), bulk-archive (NEW), bulk-export (NEW — to xlsx).

- EditMemberDialog: add "Brand asset inheritance" section — let admin override the member's chapter brand for their member-portal view (OPTIONAL, low priority).

4.5 Speaker manager — `/admin/speakers` (existing, extend)

Existing. Scoped via `scopeChapterWhere`. Has chapterId.

Extend:

- Country-tier fallback: a Country Admin sees all speakers in their country's chapters (already works via `scopeChapterWhere` which uses `chapter.countryId`).
- Cross-chapter speakers (a speaker who spoke at events in multiple chapters): the Speaker row is per-event, so the same person can have multiple Speaker rows. Add a "Group by person" toggle that dedupes by `userId` (or `contactEmail`) and shows all their Speaker rows.
- Add "Country" column (currently shows "Chapter" via `event.chapterRef`).
- Add bulk-assign-scope (exists).

4.6 Quiz manager — `/admin/quiz` (existing, extend)

Existing. Lists all quiz sessions globally (NOT scoped — gap). QuizSession has no `chapterId` today.

Fix + extend:

- Add `chapterId` to QuizSession (Section 1.2.1).
- Update `/admin/quiz` page query: `const where = { ...scopeQuizWhere(scope) };`.
- Update `/api/admin/quiz` GET to scope.
- Update `/api/admin/quiz` POST to write `chapterId` from the linked event (or from the caller's scope if no event).
- Country Admin can see all sessions in their country's chapters; Chapter Organizer sees only their chapter's sessions.
- NEW: Quiz content library — reusable `QuizQuestion` templates scoped to chapter/country/global. (Out of scope for this plan; deferred to V8.)

4.7 Testimonial manager — `/admin/testimonials` (existing, fix bug + extend)

Existing. Has the `me.role !== "ADMIN"` bug (excludes SUPER_ADMIN). NOT scoped.

Fix + extend:

- Fix the role gate: `if (!can(me.role, "testimonials.moderate")) redirect("/events")` (new permission, granted to ADMIN+ + CHAPTER_ORGANIZER).
- Add `chapterId` to Testimonial (Section 1.2.2).
- Update page query: `const where = { hidden: false, ...scopeTestimonialWhere(scope) };`.
- Update `/api/testimonials` (public GET) to accept `?chapter=slug` and filter accordingly.
- Update `/api/testimonials/[id]` PATCH/DELETE to scope-check the caller.
- Add "Country" column. Add tier filter chips.

4.8 Event manager — `/admin/events` (existing, verify scope works end-to-end)

Existing. Scoped via `scopeEventWhere`. Has `chapterId` + `isCrossChapter`.

Verify:

- `/api/admin/events` GET scopes via `scopeEventWhere(scope)`. •
- `/api/admin/events` POST validates `chapterId` against caller's scope. •
- `/api/admin/events/[id]` PATCH scope-checks. •
- `/api/admin/events/[id]` DELETE — Super Admin only. •
- Public `/api/events` (no scope — public). •
- Public `/api/events/[slug]` — single event, no scope needed. •

Extend:

- Per-event chapter branding on public page (`/e/[slug]`): use `event.chapterRef` to resolve branding.
- Cross-chapter events: when `isCrossChapter=true`, the event appears in every chapter's `/c/[chapterSlug]` event list within the same country. Verify the resolver.

4.9 Chapter settings — `/admin/chapters/[id]` (existing, verify + extend)

Existing. Chapter editor with name, slug, country, city, timezone, WhatsApp, LinkedIn, hero image. Verified.

Extend:

- Add "Brand assets" section (links to `/admin/branding/chapters/[id]`).
- Add "Email defaults" section — per-chapter `fromName`, `fromEmail`, `replyTo` overrides (stored in `ChapterSetting` keys `emailFromName`, `emailFromEmail`, `emailReplyTo`).
- Add "Locale" field — `Chapter.locale` (new column).
- Add "Cross-chapter events visible here?" toggle (always true for now; future: chapter admin can opt-out).

4.10 Country settings — `/admin/countries` (existing, extend)

Existing. Country manager (Super Admin only) with name, code, slug, flag, email defaults.

Extend:

- Add "Brand assets" section (links to `/admin/branding/countries/[id]`).
- Add "Default timezone" + "Default locale" fields.
- Add "Chapters in this country" list with quick-edit links.
- Add "Country admin" assignment (link to a User with `role=ADMIN, countryId=this.id`).

4.11 Global settings — `/admin/global` (NEW, Super Admin only)

Layout:

- "Global brand assets" section (links to `/admin/branding/global`).
- "Global email defaults" section — `SiteSetting[emailFromName]`, `SiteSetting[emailFromEmail]`, `SiteSetting[emailReplyTo]` (new keys).

- "Feature flags" section — toggle `EMAIL_PER_TIER_ENABLED`, `V7_TIER_ENFORCEMENT`, `SCOPE_SWITCHER_ENABLED` (env-var overrides, stored in `SiteSetting` so they can be flipped without redeploy).
- "Super Admin emails" section — read-only list of `SUPER_ADMIN_EMAILS` (with a note that adding requires code edit + redeploy; future: DB-driven allowlist).
- "Backup / restore" section — link to existing `/api/admin/backup-db`.

Section 5 — Email System Completion

5.1 Per-tier from-address resolution

```
async function resolveFromEmail(chapterId?: string, countryId?: string): Promise<{ fromName:
 string; fromEmail: string; replyTo: string }> {
  // 1. Chapter override (ChapterSetting keys emailFromName / emailFromEmail / emailReplyTo)
  if (chapterId) {
    const settings = await db.chapterSetting.findMany({ where: { chapterId, key: { in: ["ema
ilFromName", "emailFromEmail", "emailReplyTo"] } } });
    const map = Object.fromEntries(settings.map(s => [s.key, s.value]));
    if (map.emailFromName && map.emailFromEmail && map.emailReplyTo) {
      return { fromName: map.emailFromName, fromEmail: map.emailFromEmail, replyTo: map.emai
lReplyTo };
    }
  }
  // 2. Country default (Country.defaultFromName / defaultEmailDomain / defaultReplyTo)
  if (countryId) {
    const country = await db.country.findUnique({ where: { id: countryId } });
    if (country?.defaultFromName && country?.defaultEmailDomain && country?.defaultReplyTo)
{
      return {
        fromName: country.defaultFromName,
        fromEmail: `noreply@${country.defaultEmailDomain}`,
        replyTo: country.defaultReplyTo,
      };
    }
  }
  // 3. Global default (SiteSetting keys)
  const globalSettings = await db.siteSetting.findMany({ where: { key: { in: ["emailFromName
", "emailFromEmail", "emailReplyTo"] } } });
  const gmap = Object.fromEntries(globalSettings.map(s => [s.key, s.value]));
  if (gmap.emailFromName && gmap.emailFromEmail && gmap.emailReplyTo) {
    return { fromName: gmap.emailFromName, fromEmail: gmap.emailFromEmail, replyTo: gmap.ema
ilReplyTo };
  }
  // 4. Env-var fallback (legacy)
  const adminEmail = process.env.ADMIN_EMAIL || "eze@massapro.com";
  return { fromName: "AI Salon", fromEmail: adminEmail, replyTo: adminEmail };
}
```

5.2 Per-tier template inheritance

```
async function resolveStageTemplate(stageTemplateId: string, chapterId?: string, countryId?:
 string) {
  // 1. Chapter override (TierEmailTemplateOverride tier=chapter)
  // 2. Country override (TierEmailTemplateOverride tier=country)
  // 3. EmailStageTemplate with chapterId = this chapter (per-chapter template)
  // 4. EmailStageTemplate with chapterId IS NULL (global default)
  // Returns: { subject, htmlBody, logoUrl, fromName, fromEmail, replyTo, altSubject, altNot
OpenedHours, noCodeHtmlBody, noCodeSubject }
}
```

5.3 Admin UI flow for managing templates at each tier

- Super Admin at `/admin/email/templates` — sees all 5 global stage templates + can edit them + can create new global custom templates.
- Super Admin switches scope to "Israel" → sees the same 5 templates + a "Country override" column with "Edit override" buttons (creates a `TierEmailTemplateOverride[tier=country, countryId=IL]` row).
- Super Admin switches scope to "Tel Aviv" → sees the 5 templates + "Chapter override" column + the country override (read-only).
- Country Admin at `/admin/email/templates` — sees the 5 global templates (read-only) + "Country override" column (editable for their country) + "Chapter override" column (editable for any chapter in their country).
- Chapter Organizer at `/admin/email/templates` — sees the 5 global templates (read-only) + the country override (read-only) + "Chapter override" column (editable for their chapter only).

5.4 Sender-side code changes

Files to modify:

1. `src/lib/email-campaign/sender.ts` — replace hard-coded `fromName = "AI Salon Tel Aviv"` with `resolveFromEmail(campaign.chapterId, campaign.chapter?.countryId)`.
2. `src/app/api/admin/email/campaigns/[id]/send/route.ts` — same fix.
3. `src/app/api/cron/email/route.ts` — same fix.
4. `src/lib/email-orchestrator/worker.ts` — when sending a stage email, call `resolveStageTemplate(stageTemplateId, queue.chapterId, queue.chapter?.countryId)` instead of reading `EmailStageTemplate` directly.
5. `src/lib/email-orchestrator/templates.ts` — update the template resolver to walk the tier chain.
6. `src/app/api/email/unsubscribe/route.ts` — replace hard-coded "AI Salon Tel Aviv mailing list" with `chapter.name + " mailing list"`.
7. `src/app/api/speakers/[id]/messages/route.ts` — wire `getRelayRecipientsForEvent(eventId)` instead of `process.env.ADMIN_EMAIL`. Use `resolveFromEmail(chapterId, countryId)` for the relay email.
8. `src/app/api/messages/[userId]/route.ts` — wire `getRelayRecipientsForDM(senderId)` + per-tier from-address.

5.5 Bounce / compliance handling per chapter

- Each chapter's `replyTo` inbox should be polled by `/api/cron/email/imap-poll`. Today it polls a single inbox (`ADMIN_EMAIL`). Extend to poll multiple inboxes (one per chapter's `replyTo` address) — or use a single inbox with chapter-routing via `+chapter@` suffix (e.g. `noreply+tel-aviv@aisalon.org`).
- Unsubscribe (`/api/email/unsubscribe`) — store the unsubscribe at the `(recipientEmail, chapterId)` level, not globally. A user unsubscribed from Tel Aviv should still receive Montreal emails. New model: `EmailUnsubscribe(email, chapterId, reason, createdAt)` with `@@unique([email, chapterId])`.
- Bounce handling — `/api/email/open` and click tracking already work per-recipient via `trackToken`. Bounce webhooks (if using a service like SendGrid) should mark `EmailRecipient.status=BOUNCED` + add to `EmailUnsubscribe` for that chapter.

Section 6 — Asset Storage & CDN Strategy

6.1 Path convention for tier-scoped assets

Vercel Blob paths (extend the existing 6 conventions):

Tier	Path	Example
`Global`	`brand-assets/global/<kind>/<filename>`	`brand-assets/global/logo/abc123.png`
`Country`	`brand-assets/countries/<countryId>/<kind>/<filename>`	`brand-assets/countries/il/favicon/def456.ico`
`Chapter`	`brand-assets/chapters/<chapterId>/<kind>/<filename>`	`brand-assets/chapters/tel-aviv/banner/ghi789.jpg`
Chapter hero (existing)	`chapter-hero/<chapterId>/<filename>`	(keep — back-compat)
Chapter brand (existing)	`chapter-brand/<chapterId>/<filename>`	(keep — back-compat; will be deprecated)
Event image (existing)	`events/<eventId>/<filename>`	(keep)
Event presentation (existing)	`events/<eventId>/presentations/<filename>`	(keep)
Testimonial (existing)	`testimonials/<filename>`	(keep — but add `testimonials/<chapterId>/<filename>` for new uploads)
Creative (NEW)	`creatives/<tier>/<tierId>/<filename>`	`creatives/chapter/tel-aviv/jkl012.png`
Member photo (existing)	`member-photos/<userId>/<filename>`	(keep)
Speaker photo (existing)	`speaker-photos/<speakerId>/<filename>`	(keep — inferred from code)

Existing `brand-assets/<filename>` (flat) uploads continue to work; new uploads use the tier-prefixed convention. No migration of existing blobs (just metadata backfill into `BrandAsset` table).

6.2 Upload API changes

Every upload API route accepts additional params:

- `tier` — "global" | "country" | "chapter" (defaults to caller's natural scope)
- `countryId` — required when `tier=country` or `tier=chapter` (validated against caller's scope)
- `chapterId` — required when `tier=chapter`
- `kind` — "logo" | "favicon" | "heroLogin" | "heroChapter" | "banner" | "ogImage" | "mockup" | "socialPost" | "adAsset" | "emailHeader" | "linkedinCard" | "creative"

The route:

1. Validates the caller's scope allows uploading to the requested tier (Chapter Organizer cannot upload to country tier; Admin cannot upload to global tier).
2. Uploads the file to Vercel Blob at the tier-prefixed path.
3. Creates a `BrandAsset` row (for brand assets) or `Creative` row (for creatives).
4. Returns the blob URL + the row ID.

6.3 Signed URL / access control

- **Brand assets** (logo, favicon, hero, banner, OG) — PUBLIC. They're meant to be served to every visitor. Vercel Blob's default public URL is fine.
- **Creatives** (mockups, ads, social posts) — PUBLIC by default. Marked `isPrivate` per-row if needed (future).
- **Member photos, speaker photos** — PUBLIC (visible in community grid, event pages).
- **Event images, presentations** — PUBLIC for images; presentations are PUBLIC (members-only access enforced at the page level, not the blob).
- **Backup files** (`backups/`) — PRIVATE (Super Admin only). Use Vercel Blob's signed URL with short TTL.

No signed URL changes needed for the tier-scoped assets (they're all public).

6.4 CDN caching strategy

- Vercel Blob serves with `Cache-Control: public, max-age=31536000, immutable` (one year) by default. Brand assets get the same.
- For inheritable assets (chapter inherits country logo), the resolver walks the tier chain on every request. To avoid 3 DB queries per page load, cache the resolution in-memory (Node.js process) with a 60-second TTL keyed by `(kind, chapterId, countryId)`. Invalidate on `BrandAsset` write.
- Next.js `fetch()` cache: for public routes, use `cache: 'force-cache'` + revalidate every 60 seconds.
- Image optimization: use `next/image` with `loader: 'vercel'` (default). Brand assets get the same optimization as event images.

6.5 Migration of existing `brand-assets/` uploads

- Keep existing flat `brand-assets/<filename>` URLs as-is (they're referenced in `SiteSetting` + `ChapterSetting` rows).
- The `backfill-brand-asset-table.ts` script reads existing rows + inserts equivalent `BrandAsset` rows. The original blob URLs are preserved (no re-upload).
- New uploads use the tier-prefixed convention. Old URLs continue to resolve.

6.6 Cleanup / orphan policy

- When a chapter sets `inheritFromParent=true` on a brand asset, the previously-uploaded blob is NOT deleted (it might be referenced elsewhere). It's just unassigned.
- When a chapter deletes a `BrandAsset` row, the blob is deleted from Vercel Blob (only if no other `BrandAsset` row references the same URL — defensive against double-reference).
- When a `Creative` is deleted, the blob is deleted (only if no `BrandAsset` references it).
- When a chapter is deleted (`DELETE /api/admin/chapters/[id]`), all its `BrandAsset` rows are deleted (cascade), but the blobs are NOT deleted (orphan-keep policy — admin can manually clean up via a future `/admin/orphan-blobs` page).

Section 7 — Timezone & Localization

7.1 Per-chapter timezone field

- **Already exists.** `Chapter.timezone String @default("Asia/Jerusalem")`. Verified at `prisma/schema.prisma:59`.
- **NEW.** `Country.defaultTimezone String?` + `Chapter.locale String?` + `Country.defaultLocale String?` (Section 1.4).

7.2 Central `getChapterTimezone(chapterId)` helper

New file `src/lib/chapter-tz.ts`:

```
import { db } from "@/lib/db";

const FALLBACK_TZ = "Asia/Jerusalem"; // legacy default

export async function getChapterTimezone(chapterId: string | null | undefined): Promise<stri
ng> {
  if (!chapterId) return FALLBACK_TZ;
  const chapter = await db.chapter.findUnique({
    where: { id: chapterId },
    select: { timezone: true, country: { select: { defaultTimezone: true } } },
  });
  return chapter?.timezone || chapter?.country?.defaultTimezone || FALLBACK_TZ;
}

export async function getChapterLocale(chapterId: string | null | undefined): Promise<string
> {
  if (!chapterId) return "en-US";
  const chapter = await db.chapter.findUnique({
    where: { id: chapterId },
    select: { locale: true, country: { select: { defaultLocale: true } } },
  });
  return chapter?.locale || chapter?.country?.defaultLocale || "en-US";
}

// Synchronous formatter that takes the timezone string directly
export function formatInTz(date: Date, tz: string, opts: Intl.DateTimeFormatOptions): string
 {
  try {
```

```
    return new Intl.DateTimeFormat("en-US", { ...opts, timeZone: tz }).format(date);
  } catch {
    return new Intl.DateTimeFormat("en-US", opts).format(date);
  }
}
```

7.3 Replace hard-coded `Asia/Jerusalem` (15 file:line refs)

Files to update (from EXPLORE-1 inventory section 7):

1. `src/app/events/events-list.tsx:46-73` — `timeZone: "Asia/Jerusalem"` in Intl.DateTimeFormat. Accept `tz` prop from server component.
2. `src/app/events/[slug]/page.tsx:442-515` — same. Pass `chapter.timezone` from the event's chapter.
3. `src/app/events/[slug]/tabs/{overview,agenda,admin-agenda,presentations,photos}-tab.tsx` — same. Pass `tz` from props.
4. `src/app/events/my-registered-events.tsx:29,38` — same.
5. `src/app/admin/events/new/event-creator.tsx:209,213,543,550,557` — defaults `"Tel Aviv"` / `"ISR"`. Replace with `selectedChapter?.city ?? ""` / `selectedChapter?.country?.code ?? ""`.
6. `src/app/admin/events/new/new-event-form.tsx:85,86,295` — `useState(defaultChapter?.city ?? "Tel Aviv")`. Replace `"Tel Aviv"` with empty string.
7. `src/app/admin/chapters/route.ts:24` — `timezone ?? "Asia/Jerusalem"`. Keep as fallback (it's only used if the admin doesn't provide a timezone).
8. `src/app/api/admin/events/route.ts:62` — `chapter || "Tel Aviv"`. Replace with `chapter || ""`.
9. `src/app/api/admin/events/[id]/route.ts:173` — same.
10. `src/app/api/admin/events/extract/route.ts:54,66,69` — LLM system prompt hard-codes "AI Salon Tel Aviv" + defaults city/timezone. Replace with the chapter's values (looked up from the request body's `chapterId`).
11. `prisma/schema.prisma:55` — `Chapter.timezone String @default("Asia/Jerusalem")`. Keep as default for new chapters (Israel is the first chapter).
12. `prisma/schema.prisma:316` — `Event.chapter String @default("Tel Aviv")`. Mark deprecated. Phase 7: drop the column.

7.4 Per-chapter display name (replace hard-coded "Tel Aviv Chapter")

Files to update (from EXPLORE-1 inventory section 7):

1. `src/app/layout.tsx:53,54,57-65,68,84` — metadata title template `%s — AI Salon Tel Aviv`. Replace with a `generateMetadata()` that reads the route's chapter context and returns `%s — AI Salon ${chapter.name}` or `%s — AI Salon` (global).
2. `src/components/ais/app-header.tsx:71` — `<span>Tel Aviv Chapter</span>`. Replace with `<span>{chapter?.name ?? 'AI Salon'} Chapter</span>` (chapter resolved from the current route).
3. `src/app/login/page.tsx:40-162` — "AI Salon Tel Aviv" / "Tel Aviv Chapter" / "Empowering AI Connections in Tel Aviv". Replace with chapter-aware strings resolved from `?chapterSlug=`.

4. `src/app/onboarding/page.tsx:60,72,105` — same.
5. `src/app/api/email/unsubscribe/route.ts:58,60` — "AI Salon Tel Aviv mailing list". Replace with `chapter.name + " mailing list"`.
6. `src/app/api/cron/email/route.ts:97,175` — `fromName = ... || "AI Salon Tel Aviv"`. Replace with `resolveFromEmail(...).fromName`.
7. `src/app/api/admin/email/campaigns/[id]/send/route.ts:79` — same.
8. `src/app/api/speakers/[id]/messages/route.ts:153` — footer `— AI Salon Tel Aviv platform`. Replace with `— AI Salon ${chapter.name}`.
9. `src/app/api/messages/[userId]/route.ts:189,195,199,209` — DM relay email. Same.
10. `src/app/profile/page.tsx:56,75` + `src/app/onboarding/onboarding-form.tsx:103` + `src/app/events/[slug]/page.tsx:606` + `src/app/admin/page.tsx:290` — footer `© AI Salon Tel Aviv · Empowering AI Connections`. Replace with `© AI Salon ${chapter?.name ?? ''}` or just `© AI Salon`.
11. `src/app/admin/{mockups/*,email/*,members/*}` — many comments + UI strings. Replace with chapter-aware strings.

7.5 i18n strategy

Decision: minimal i18n rollout. Wire `next-intl` (already installed) for UI string translation only — NOT for content translation (event descriptions stay single-language).

- Create `messages/en-US.json`, `messages/he-IL.json`, `messages/fr-CA.json` with UI strings (button labels, page titles, footer text).
- Add `NextIntlClientProvider` in `src/app/layout.tsx` with the chapter's locale.
- Replace hard-coded UI strings with `useTranslations()` calls.
- Content (event descriptions, email templates, testimonials) stays as single-language strings in the DB.

Rationale. Full content i18n would require a `Translation` table + per-field locale resolution + admin UI for translators — out of scope. UI i18n gives Montreal French buttons + Tel Aviv Hebrew buttons without DB schema bloat. Content translation can be added in V8 if needed.

Recommendation. Ship Phase 7 (Section 10) with UI i18n for en-US only. Add he-IL + fr-CA in a follow-up. Don't block the tier-completion work on i18n.

7.6 Per-chapter locale

- `Chapter.locale String?` — defaults to `Country.defaultLocale` which defaults to `"en-US"`.
- Used by: `NextIntlClientProvider` (UI strings), `Intl.DateTimeFormat` (date formatting — locale affects weekday names, AM/PM), `Intl.NumberFormat` (number formatting).
- Tel Aviv: `he-IL` (or `en-US` if the chapter prefers English). Montreal: `fr-CA` (or `en-CA`). Default: `en-US`.

Section 8 — API Layer Changes

8.1 Routes that already use `scopeWhere()` — verify tier context

These routes already scope via `scopeUserWhere` / `scopeEventWhere` / `scopeChapterWhere`:

Route	Scope helper	Verification
`GET /api/admin/chapters`	`getUserScope` + `canActOnChapter`	• verified
`GET /api/admin/countries`	`getUserScope`	•
`GET /api/admin/events`	`scopeEventWhere`	•
`GET /api/admin/registrants`	`scopeChapterWhere` + `getCoHostedEventIds`	•
`GET /api/admin/speakers`	`scopeChapterWhere`	•
`GET /api/admin/email/campaigns`	`emailModelWhere` (custom)	•
`GET /api/admin/email/templates`	`emailModelWhere`	•
`GET /api/admin/email/flows`	`emailModelWhere`	•
`GET /api/admin/email/audiences`	`emailModelWhere`	•
`GET /api/email-orchestrator/queue`	`scopeChapterWhere`	•
`GET /api/admin/analytics`	`scopeChapterWhere`	•
`GET /api/admin/chapters/for-assign`	scope-aware	•

8.2 Routes that need to START using `scopeWhere()`

These routes do NOT scope today (inventory gaps):

Route	Current	Fix
`GET /api/admin/members`	Returns ALL members	`const where = { archivedAt: null, ...scopeUserWhere(scope) };`
`GET /api/admin/quiz`	Returns ALL sessions	`const where = { ...scopeQuizWhere(scope) };` (after adding `chapterId` to QuizSession)
`GET /api/admin/quiz/[id]/results`	No scope check	Verify the quiz's chapter is in caller's scope before returning results
`GET /api/admin/testimonials` (if exists)	`N/A — testimonials are public GET, admin-moderated via PATCH`	Add scope check to PATCH
`GET /api/admin/non-members`	Returns ALL non-member RSVPs	Scope via `scopeChapterWhere`
`GET /api/admin/hidden-images`	Returns ALL hidden images	No scope needed (admin-only read)
`GET /api/chat/rooms`	Returns ALL rooms the user is in	Filter by `userId` (existing) — but for admin moderation, add `GET /api/admin/chat/rooms` scoped via `scopeChatWhere`
`GET /api/messages/conversations`	Returns ALL the user's conversations	No scope needed (per-user)

8.3 New routes needed

Route	Purpose
`GET /api/admin/scope`	Returns the caller's current effective scope (natural + cookie override)
`POST /api/admin/scope`	Sets the scope override cookie
`DELETE /api/admin/scope`	Clears the scope override (resets to natural)
`GET /api/admin/branding?tier=X&countryId=Y&chapterId=Z`	List `BrandAsset` rows for a tier
`POST /api/admin/branding`	Create/update a `BrandAsset` row
`DELETE /api/admin/branding/[id]`	Delete a `BrandAsset` row (revert to parent)
`POST /api/admin/branding/upload`	Multipart upload to Vercel Blob + return URL
`GET /api/admin/creatives?tier=X&countryId=Y&chapterId=Z&kind=K`	List `Creative` rows (scoped)
`POST /api/admin/creatives`	Upload a creative (multipart + create row)
`PATCH /api/admin/creatives/[id]`	Edit creative metadata
`DELETE /api/admin/creatives/[id]`	Delete a creative (+ delete blob)
`POST /api/admin/creatives/[id]/assign`	Assign a creative as a `BrandAsset` for a tier
`GET /api/admin/email/templates/overrides?tier=X&countryId=Y&chapterId=Z`	List `TierEmailTemplateOverride` rows
`POST /api/admin/email/templates/overrides`	Create/update an override
`DELETE /api/admin/email/templates/overrides/[id]`	Delete an override
`GET /api/admin/global/settings`	Super Admin — global defaults (email, brand, feature flags)
`PATCH /api/admin/global/settings`	Super Admin — update global defaults
`GET /api/admin/countries/[id]/branding`	Country brand assets (alias for `/api/admin/branding?tier=country&id=[id]`)
`POST /api/admin/countries/[id]/branding`	Upload country brand asset
`GET /api/admin/chapters/[id]/branding`	Chapter brand assets (alias)
`POST /api/admin/chapters/[id]/branding`	Upload chapter brand asset
`GET /api/admin/countries/[id]/chapters`	List chapters in a country (scoped)

8.4 Routes that need a `tier` query param

List endpoints that currently return all rows in the caller's scope should accept `?tier=global|country|chapter&countryId=X&chapterId=Y` to filter the displayed tier. Useful for the admin UI when the Super Admin has switched scope to a specific chapter but wants to see "show me only global templates" within that view.

Affected routes:

- `GET /api/admin/email/campaigns?tier=X`

- `GET /api/admin/email/templates?tier=X`
- `GET /api/admin/email/flows?tier=X`
- `GET /api/admin/email/audiences?tier=X`
- `GET /api/admin/creatives?tier=X`
- `GET /api/admin/branding?tier=X`

The `tier` param is OPTIONAL. When omitted, the route returns rows in the caller's effective scope (chapter + country + global for inheritable assets; chapter + country for non-inheritable).

8.5 `withScope()` middleware wrapper

New file `src/lib/with-scope.ts`:

```
import { NextRequest, NextResponse } from "next/server";
import { getCurrentUser } from "@/lib/auth-guards";
import { getEffectiveScope, UserScope } from "@/lib/permissions";

type ScopedHandler = (
  req: NextRequest,
  ctx: { params: Record<string, string>; user: User; scope: UserScope }
) => Promise<NextResponse>;

export function withScope(handler: ScopedHandler): (req: NextRequest, ctx: { params: Record<
string, string> }) => Promise<NextResponse> {
  return async (req, ctx) => {
    const { user, error } = await getCurrentUser();
    if (error || !user) return NextResponse.json({ error: "Unauthorized" }, { status: 401 })
;
    const scope = await getEffectiveScope(user.id, req.cookies.get("scope")?.value);
    if (scope.kind === "none") return NextResponse.json({ error: "Forbidden" }, { status: 40
3 });
    return handler(req, { params: ctx.params, user, scope });
  };
}
```

Usage:

```
export const GET = withScope(async (req, { user, scope }) => {
  const where = { archivedAt: null, ...scopeUserWhere(scope) };
  const members = await db.user.findMany({ where });
  return NextResponse.json({ members });
});
```

Every admin list endpoint should use `withScope()`. The wrapper:

1. Authenticates the user (rejects if not signed in).
2. Resolves the effective scope (natural + cookie override).
3. Rejects if scope is "none" (no admin access).
4. Passes `user` + `scope` to the handler.

This eliminates the per-route `getCurrentUser()` + `getUserScope()` boilerplate and ensures every scoped route has the scope available.

Section 9 — Migration & Rollout Plan

9.1 Migration order

1. **Phase 1 (schema additive).** Run the new migration: adds `chapterId` to 11 tables (QuizSession, Testimonial, ChatRoom, EventImage, PresentationFile, EventMockupDefault, SpeakerMessage, ConversationMessage, MemberTag, EventPrepQuestion, EventPrepSuggestion, EmailEvent, TrackingLog) + creates `BrandAsset` + `Creative` tables + renames `ChapterEmailTemplateOverride` to `TierEmailTemplateOverride` + adds columns to `Country` + `Chapter`. All additive; no drops. ZERO downtime.
2. **Phase 2 (backfill).** Run `scripts/backfill-tier-chapter-ids.ts` — for every new `chapterId` column, backfill from the parent (event → chapter, speaker → chapter, etc.). Idempotent. ZERO downtime.
3. **Phase 3 (backfill BrandAsset).** Run `scripts/backfill-brand-asset-table.ts` — reads existing `SiteSetting` + `ChapterSetting` + `Chapter.heroImageUrl` + `EmailStageTemplate.logoUrl` + `EventMockupDefault.imageUrl` and inserts equivalent rows into `BrandAsset` / `Creative`. Idempotent. ZERO downtime.
4. **Phase 4 (code deploy).** Push the new code to Vercel. New scope helpers + new admin UI + new API routes go live. Existing routes continue to work (they don't read the new columns yet). ZERO downtime.
5. **Phase 5 (read-path enforcement).** Flip the `V7_TIER_ENFORCEMENT` feature flag from "off" to "on". Now `withScope()` rejects unauthenticated requests to admin routes. Existing admin pages start filtering by the new `chapterId` columns. Possible brief spike in 403s if any route is missing the scope helper — monitor Sentry.
6. **Phase 6 (write-path enforcement).** Flip `V7_TIER_ENFORCEMENT_WRITES` from "off" to "on". Now POST/PATCH/DELETE routes validate the caller's scope against the row's `chapterId`. Possible brief spike in 403s if any write route is missing the check.
7. **Phase 7 (cleanup).** Drop the legacy `Event.chapter String` column. Drop the `src/lib/v7-scope.ts` dead-code file. Drop `src/lib/admin-auth.ts` (legacy). Drop the duplicate `/api/admin/events/[id]/cohosts/` route (legacy hyphenless variant). This phase IS breaking — coordinate with a maintenance window.

9.2 Backfill strategy

- All backfills are idempotent (`UPDATE ... WHERE chapterId IS NULL`).
- All backfills run server-side via `npx tsx scripts/...` against the production DB (Neon Postgres).
- The `/api/admin/v7-seed` endpoint already runs the original V7 backfill (Israel/Tel Aviv). Extend it to also run the new backfills (Phase 2 + Phase 3) so the Super Admin can trigger from the UI.
- For legacy rows that have NO parent (e.g. a `Testimonial` with no `eventId` AND no `speakerId` AND no `authorId` chapter), backfill to Israel/Tel Aviv (the default chapter) — same as the original V7 seed.

9.3 Zero-downtime considerations

- Vercel deployments are atomic (the new code goes live all at once). But DB migrations need care:
  - Phase 1 (schema additive): safe to run during traffic. `ALTER TABLE ADD COLUMN` on Postgres takes a brief lock; for large tables (User with ~10K rows, EventRsvp with ~50K rows), the lock is sub-second. No downtime.
  - Phase 2 (backfill UPDATEs): for large tables, run in batches of 1000 rows with a 100ms sleep between batches to avoid locking. The `backfill-tier-chapter-ids.ts` script supports `--batch-size=1000`

`--sleep=100`.

- Phase 3 (BrandAsset backfill): small dataset (~50 rows). No batching needed.
- Phase 4 (code deploy): Vercel handles the swap. No downtime.
- Phase 5 + 6 (flag flips): no DB change. Just env-var updates via Vercel dashboard.
- Phase 7 (cleanup): `ALTER TABLE DROP COLUMN` on Postgres takes a brief lock. For the `Event.chapterString` column, the lock is sub-second. Coordinate with a low-traffic window (Sunday 02:00 UTC) just in case.

9.4 Feature-flag strategy

Feature flags stored in `SiteSetting` (so they can be flipped without redeploy):

Flag	Default	Effect when "off"	Effect when "on"
`V7_TIER_ENFORCEMENT`	off	Admin routes skip `withScope()` (legacy behavior)	Admin routes require scoped auth
`V7_TIER_ENFORCEMENT_WRITES`	off	Write routes skip scope-check on `chapterId`	Write routes validate scope
`EMAIL_PER_TIER_ENABLED`	off	Email sender uses `ADMIN_EMAIL` env-var	Email sender uses `resolveFromEmail(chapterId, countryId)`
`SCOPE_SWITCHER_ENABLED`	off	Scope switcher hidden in UI	Scope switcher visible
`BRAND_ASSET_TABLE_ENABLED`	off	Brand resolver reads `SiteSetting` / `ChapterSetting` (legacy)	Brand resolver reads `BrandAsset` table

Rollout per flag:

- `BRAND_ASSET_TABLE_ENABLED`: flip first (after Phase 3 backfill). Lowest risk — only affects brand-image resolution.
- `SCOPE_SWITCHER_ENABLED`: flip second. UI-only, no enforcement.
- `EMAIL_PER_TIER_ENABLED`: flip third. Requires chapter/country email defaults to be set first (Super Admin configures via `/admin/branding/global` + `/admin/branding/countries/[id]`).
- `V7_TIER_ENFORCEMENT`: flip fourth. Read-path enforcement.
- `V7_TIER_ENFORCEMENT_WRITES`: flip fifth. Write-path enforcement.

9.5 Rollback plan per phase

Phase	Rollback action
1 (schema)	`ALTER TABLE ... DROP COLUMN ...` for each new column. Or restore from DB backup (Neon PITR — point-in-time recovery).
2 (backfill)	`UPDATE ... SET chapterId = NULL` for each backfilled column. Or restore from backup.
3 (BrandAsset backfill)	`DELETE FROM "BrandAsset"; DELETE FROM "Creative";` — no impact on existing code (the table is unused until `BRAND_ASSET_TABLE_ENABLED=on`).
4 (code deploy)	Vercel auto-rollback to previous deployment (one click in dashboard).
5 (read-path flag)	Flip `V7_TIER_ENFORCEMENT=off` in Vercel env-vars.
6 (write-path flag)	Flip `V7_TIER_ENFORCEMENT_WRITES=off`.
7 (cleanup)	Cannot rollback (column dropped). Restore from DB backup.

9.6 Testing strategy

- **Per-tier integration tests.** For each content type, create two chapters (`test-chapter-a`, `test-chapter-b`) + a country (`test-country`) + data in each. Log in as each role (SUPER_ADMIN, ADMIN of test-country, CHAPTER_ORGANIZER of test-chapter-a). Assert:
  - SUPER_ADMIN sees all rows.
  - ADMIN sees rows in test-country's chapters only.
  - CHAPTER_ORGANIZER sees rows in test-chapter-a only.
  - MEMBER sees no rows (403).
- **Scope-leak tests.** Specifically: CHAPTER_ORGANIZER of test-chapter-a POSTs to create a row with `chapterId=test-chapter-b` — assert 403. ADMIN of test-country POSTs with `chapterId=other-country-chapter` — assert 403.
- **Inheritance tests.** Set `BrandAsset[tier=global, kind=logo] = "logo-A.png"`. Set `BrandAsset[tier=country, kind=logo, inheritFromParent=true]`. Resolve `resolveBrandAsset("logo", chapterId-in-country)` — assert returns "logo-A.png". Set `inheritFromParent=false, value="logo-B.png"`. Resolve — assert returns "logo-B.png".
- **Email tier tests.** Set `Country.defaultFromName = "AI Salon Israel"`. Send an email campaign with `chapterId=tel-aviv`. Assert the sent email's `fromName` = "AI Salon Israel" (no chapter override). Set `ChapterSetting[chapterId=tel-aviv, key=emailFromName, value="AI Salon Tel Aviv"]`. Send again — assert `fromName` = "AI Salon Tel Aviv".
- **Timezone tests.** Create an event in `Chapter(timezone="America/Montreal")` with `startsAt = 2026-08-15T18:00:00Z`. GET `/e/[slug]` — assert the displayed time is "2026-08-15 14:00:00 EDT" (Montreal time), NOT "2026-08-15 21:00:00 IDT" (Tel Aviv time).
- **E2E smoke test.** After each phase, run the existing `core/qa/smoke-tests.md` script + a new `scripts/e2e-tier-smoke.ts` that walks through every admin page as each role.

Section 10 — Implementation Phases (sequenced)

Each phase is independently shippable. Phases can be deployed to Vercel without the next phase being complete.

Phase 1 — Schema additive migration + backfill

Goal. Add `chapterId` to all 11 missing tables + create `BrandAsset` + `Creative` tables + add country brand columns + rename `ChapterEmailTemplateOverride` → `TierEmailTemplateOverride`.

Schema changes. `ALTER TABLE` for QuizSession, Testimonial, ChatRoom, EventImage, PresentationFile, EventMockupDefault, SpeakerMessage, ConversationMessage, MemberTag, EventPrepQuestion, EventPrepSuggestion, EmailEvent, TrackingLog. `CREATE TABLE` for BrandAsset + Creative. `ALTER TABLE Country ADD COLUMN defaultLogoUrl, defaultFaviconUrl, defaultHeroUrl, defaultBannerUrl, defaultTimezone, defaultLocale`. `ALTER TABLE Chapter ADD COLUMN locale`. `ALTER TABLE ChapterEmailTemplateOverride RENAME TO TierEmailTemplateOverride` + add `tier`, `countryId`, `fromName`, `fromEmail`, `replyTo` columns.

Code changes. Update `prisma/schema.prisma` to mirror the migration. Generate new Prisma client. Update `src/lib/permissions.ts` to add `scopeQuizWhere`, `scopeTestimonialWhere`, `scopeChatWhere`, `scopeImageWhere`, `scopeMockupWhere`, `scopeBrandAssetWhere`, `scopeCreativeWhere`, `scopeTierEmailTemplateOverrideWhere`, `getEffectiveScope(userId, override?)`.

UI changes. None.

Acceptance criteria. `prisma migrate deploy` succeeds against production Neon. `npx tsx scripts/backfill-tier-chapter-ids.ts` runs and reports "0 rows left with NULL chapterId" for all 11 tables. `npx tsx scripts/backfill-brand-asset-table.ts` runs and inserts N rows into BrandAsset + Creative. Existing admin pages continue to work (no regression).

Dependencies. None.

Phase 2 — Scope switcher UI + `withScope()` wrapper

Goal. Ship the scope switcher in `AppHeader`. Add `withScope()` wrapper. Wire `getEffectiveScope()` to read the scope cookie.

Schema changes. None.

Code changes. New `src/components/ais/scope-switcher.tsx`. New `src/lib/with-scope.ts`. New `POST /api/admin/scope` + `GET /api/admin/scope` + `DELETE /api/admin/scope`. Update `src/lib/auth-guards.ts getCurrentUser()` to read scope cookie. Update `src/lib/permissions.ts getUserScope()` → `getEffectiveScope(userId, override?)`.

UI changes. `<ScopeSwitcher />` rendered in `AppHeader`. Visible only when `SCOPE_SWITCHER_ENABLED=on`. Hidden for CHAPTER_ORGANIZER (locked scope).

Acceptance criteria. Super Admin logs in → sees switcher with full country/chapter tree. Clicks "Tel Aviv" → all admin pages re-query with chapter scope. Refreshes → scope persists (cookie). Country Admin logs in → sees switcher with only their country. Chapter Organizer logs in → no switcher.

Dependencies. Phase 1 (for the `getEffectiveScope` helper signature).

Phase 3 — Brand asset + Creative panels

Goal. Ship `/admin/branding` + `/admin/creatives` + the `BrandAsset` resolver.

Schema changes. None (BrandAsset table created in Phase 1).

Code changes. New `src/lib/brand-asset.ts` (the `resolveBrandAsset` helper). New `src/app/admin/branding/...` pages. New `src/app/admin/creatives/...` pages. New API routes (Section 8.3). Rewrite `src/lib/chapter-brand-images.ts` to read from `BrandAsset` first (gated by `BRAND_ASSET_TABLE_ENABLED` flag). Update `src/app/login/page.tsx` + `src/app/c/[chapterSlug]/page.tsx` + `src/app/layout.tsx` generateMetadata to use `resolveBrandAsset`.

UI changes. `/admin/branding/global`, `/admin/branding/countries/[id]`, `/admin/branding/chapters/[id]`, `/admin/creatives`. Link from `/admin/chapters/[id]` to `/admin/branding/chapters/[id]`. Link from `/admin/countries/[id]` to `/admin/branding/countries/[id]`.

Acceptance criteria. Super Admin uploads a logo at `/admin/branding/global` → all chapters see it (via inheritance). Country Admin uploads a logo at `/admin/branding/countries/[id]` → all chapters in that country see it (chapter overrides still win). Chapter Admin uploads a logo at `/admin/branding/chapters/[id]` → only that chapter sees it. Toggle "Use parent's logo" → falls back to parent tier. `/login?chapterSlug=tel-aviv` shows the resolved chapter logo.

Dependencies. Phase 1 (BrandAsset table). Phase 2 (scope switcher for tier navigation).

Phase 4 — Email system completion

Goal. Wire `resolveFromEmail()` + `resolveStageTemplate()` into the email sender. Ship the `TierEmailTemplateOverride` admin UI.

Schema changes. None.

Code changes. New `src/lib/email-tier-resolver.ts`. Update `src/lib/email-campaign/sender.ts`, `src/app/api/admin/email/campaigns/[id]/send/route.ts`, `src/app/api/cron/email/route.ts`, `src/lib/email-orchestrator/worker.ts`, `src/lib/email-orchestrator/templates.ts`. Update `src/app/api/email/unsubscribe/route.ts` for chapter-aware unsubscribe. New `src/app/admin/email/templates/overrides/page.tsx`. New API routes (Section 8.3). Wire `src/lib/relay-recipients.ts` into `src/app/api/speakers/[id]/messages/route.ts` + `src/app/api/messages/[userId]/route.ts`.

UI changes. New `/admin/email/templates/overrides` sub-tab. New "Email defaults" section in `/admin/branding/global` + `/admin/branding/countries/[id]` + `/admin/branding/chapters/[id]`.

Acceptance criteria. With `EMAIL_PER_TIER_ENABLED=on`: send a campaign from a chapter scope — the sent email's `fromName` resolves from chapter override → country default → global default → env-var fallback. The `replyTo` resolves the same way. The email body resolves from `TierEmailTemplateOverride` (chapter) → `TierEmailTemplateOverride` (country) → `EmailStageTemplate` (global). Speaker-message relay goes to chapter organizers (not `ADMIN_EMAIL`). DM relay goes to sender's chapter organizers.

Dependencies. Phase 1 (TierEmailTemplateOverride table). Phase 3 (branding panels for email defaults UI).

Phase 5 — Timezone + UI copy de-hardcoding

Goal. Replace all 15 `Asia/Jerusalem` references + all 30 `AI Salon Tel Aviv` references with chapter-aware helpers.

Schema changes. None (Country.defaultTimezone + Chapter.locale added in Phase 1).

Code changes. New `src/lib/chapter-tz.ts`. Update all 15 files listed in Section 7.3. Update all 30 files listed in Section 7.4. Wire `next-intl` (minimal — UI strings only, en-US at first).

UI changes. Every event-display route shows times in the chapter's timezone. Every page's title reflects the chapter's name. Every email footer reflects the chapter's name.

Acceptance criteria. Open `/e/[slug]` for a Montreal event — times show in `America/Montreal`. Open `/c/tel-aviv` — page title is "AI Salon Tel Aviv". Open `/c/montreal` — page title is "AI Salon Montreal". Open `/login?chapterSlug=montreal` — hero text says "AI Salon Montreal" (not "Tel Aviv"). Open `/events` — events list shows times in the user's chapter timezone (if signed-in) or the event's chapter timezone (if anonymous).

Dependencies. Phase 1 (Country.defaultTimezone + Chapter.locale columns).

Phase 6 — Scope enforcement (read + write paths)

Goal. Flip `V7_TIER_ENFORCEMENT=on` + `V7_TIER_ENFORCEMENT_WRITES=on`. Fix the inventory bugs (`/admin/testimonials` role gate, `/api/admin/members` GET no scope).

Schema changes. None.

Code changes. Fix `/admin/testimonials` page to use `can(me.role, "testimonials.moderate")` + `scopeTestimonialWhere(scope)`. Fix `/api/admin/members` GET to use `scopeUserWhere(scope)`. Wrap every admin list endpoint in `withScope()`. Wrap every admin write endpoint in `withScope()` + add `canActOnChapter(scope, row.chapterId)` check.

UI changes. `/admin/testimonials` visible to SUPER_ADMIN + ADMIN + CHAPTER_ORGANIZER. Tier filter chips on `/admin` (members), `/admin/events`, `/admin/speakers`, `/admin/registrants`, `/admin/email`, `/admin/quiz`, `/admin/testimonials`.

Acceptance criteria. CHAPTER_ORGANIZER of Tel Aviv cannot see Montreal's members (403 on API). ADMIN of Israel cannot edit a Canada chapter's event (403). SUPER_ADMIN can see all. All 8 acceptance tests from Section 9.6 pass.

Dependencies. Phase 1 (chapterId on all tables). Phase 2 (scope switcher).

Phase 7 — Cleanup + URL aliases + i18n

Goal. Drop legacy columns/files. Add city-root URL aliases. Wire `next-intl` for he-IL + fr-CA.

Schema changes. `ALTER TABLE "Event" DROP COLUMN "chapter";` (legacy free-form String cache — all readers migrated to `chapterRef.name` in Phase 5). Drop duplicate `/api/admin/events/[id]/cohosts/` route (legacy hyphenless).

Code changes. Delete `src/lib/v7-scope.ts`. Delete `src/lib/admin-auth.ts` (after migrating all callers to `getCurrentUser()` + `can()`). Delete `prisma/schema.prisma.bak`. New `src/app/[chapterSlug]/page.tsx` (301 redirect to `/c/[chapterSlug]`). New `src/app/[countrySlug]/page.tsx` (country landing page). Add `messages/he-IL.json` + `messages/fr-CA.json`. Add `NextIntlClientProvider` in `src/app/layout.tsx`.

UI changes. Visit `/tel-aviv` → redirects to `/c/tel-aviv`. Visit `/israel` → country landing page listing chapters. Visit `/c/montreal` with `Accept-Language: fr-CA` → UI strings in French.

Acceptance criteria. All legacy columns/files gone. City-root aliases work. he-IL + fr-CA UI strings render correctly. All Phase 1-6 acceptance tests still pass.

Dependencies. Phase 6 (all readers migrated to scoped queries; safe to drop legacy columns).

Section 11 — Agent Work Assignment

For each phase, propose the sub-agent type. Multiple agents can work in parallel on different phases if dependencies allow.

Phase	Sub-agent type	Rationale
Phase 1 (schema + backfill)	`general-purpose`	Migration SQL + backfill scripts + Prisma schema edits. Requires careful SQL + idempotent script writing. Not UI work.
Phase 1 (schema review)	`Plan`	Review the migration SQL for safety + idempotency before running against production.
Phase 2 (scope switcher UI)	`full-stack-developer`	Next.js App Router component + API route + cookie handling + auth-guards update. Core full-stack work.
Phase 2 (scope switcher styling)	`frontend-styling-expert`	Polish the dropdown — tree view, breadcrumbs, color coding.
Phase 3 (BrandAsset resolver)	`general-purpose`	The resolver is pure logic — tier-walking, caching, edge cases. Not UI.
Phase 3 (branding panels UI)	`full-stack-developer`	3 new admin pages + upload flows + API routes.
Phase 3 (branding panels styling)	`frontend-styling-expert`	Card layouts, asset preview, tier badges.
Phase 3 (creatives panel UI)	`full-stack-developer`	Grid view + filter bar + upload modal + detail drawer.
Phase 4 (email tier resolver)	`general-purpose`	Pure logic — `resolveFromEmail`, `resolveStageTemplate`, tier-walking.
Phase 4 (email sender wiring)	`full-stack-developer`	Update 6+ files in the email orchestrator + API routes.
Phase 4 (template override UI)	`full-stack-developer`	Matrix view + per-cell editor + rich text.
Phase 4 (relay-recipients wiring)	`general-purpose`	2 files to update — small but careful.
Phase 5 (timezone helper)	`general-purpose`	New `chapter-tz.ts` + `formatInTz` utility.
Phase 5 (de-hardcode strings)	`full-stack-developer`	45 file:line refs across the codebase. Mechanical but wide-reaching.
Phase 5 (i18n minimal wiring)	`full-stack-developer`	Wire `next-intl` + create `messages/en-US.json`.
Phase 6 (scope enforcement)	`full-stack-developer`	Wrap every admin route in `withScope()` + fix the 2 inventory bugs.
Phase 6 (test scaffolding)	`general-purpose`	Per-tier integration tests + scope-leak tests.

Phase	Sub-agent type	Rationale
Phase 7 (cleanup + URL aliases)	`full-stack-developer`	Drop legacy columns + add new route segments + i18n expansion.
Phase 7 (i18n translation)	`general-purpose`	Translate `messages/en-US.json` → `he-IL.json` + `fr-CA.json`.
Cross-phase (exploration)	`Explore`	When an agent needs to verify "which files reference X" before making a change.
Cross-phase (design sub-tasks)	`Plan`	When a phase needs a sub-design (e.g. "design the BrandAsset resolver API" before implementation).

Parallelization. Phases 1 + 2 can run in parallel (Phase 2 doesn't need the new schema, only the `getEffectiveScope` signature). Phases 3 + 4 can run in parallel after Phase 1 (both depend only on the schema). Phase 5 can start in parallel with Phase 3 + 4 (depends only on Phase 1's `Country.defaultTimezone` column). Phase 6 must wait for Phases 3 + 4 + 5 (needs all scoped helpers + UI in place). Phase 7 must wait for Phase 6.

Critical path. Phase 1 → Phase 6 → Phase 7. Phases 2, 3, 4, 5 are parallelizable off Phase 1.

Section 12 — Decisions Resolved

All 9 design decisions that required user input before implementation have been resolved. The decisions below are now binding inputs to the phased migration in Section 10. Each decision is mapped to the phase(s) it affects.

12.1 Chapter-prefixed URLs — keep flat events, add city-root aliases

Decision. Keep `/events/[slug]` flat (preserve SEO equity already accumulated on existing event URLs). Add `/[chapterSlug]` city-root aliases (e.g. `/tel-aviv`, `/montreal`) that 301-redirect to `/c/[chapterSlug]`. Add `/[countrySlug]` country landing pages as new top-level routes (e.g. `/israel`, `/canada`) — these are full pages, not redirects.

Rationale. Existing event URLs (the highest-traffic pages on the platform) must continue to resolve. The city-root alias gives chapters a clean marketing URL without breaking the canonical chapter landing at `/c/[chapterSlug]`. Country landing pages are additive — they aggregate all chapters in a country and provide a country-tier scope for SEO + brand identity.

Implementation impact. Phase 2 routes section is updated: `src/app/[chapterSlug]/page.tsx` is a new 301-redirect route (3 lines); `src/app/[countrySlug]/page.tsx` is a new country landing page (~120 lines, lists chapters + upcoming events). No change to `/events/[slug]`.

12.2 Chapter admin isolation — read-only cross-chapter + copy

Decision. A CHAPTER_ORGANIZER can browse other chapters' templates, flows, and audiences as READ-ONLY (for inspiration). They can save a copy of any other chapter's template/flow/audience into their own chapter. They cannot modify the original.

Rationale. Pure isolation (current V7 design) forces every chapter to build their email templates from scratch — a waste of effort when 80% of email copy is reusable across chapters. Read-only browse + copy-to-own gives chapters a shared library without compromising write isolation. The original template's `chapterId` is preserved; the copy gets the organizing chapter's `chapterId` and a new ID.

Implementation impact. Phase 2 admin pages: every list endpoint returns other-chapter rows with a `canEdit: false` flag (already-scoped queries already support this — just relax the WHERE to allow read across chapters in the same country). New API route `POST /api/admin/email/templates/[id]/copy` accepts `{ targetChapterId }`, clones the template, and returns the new ID. Phase 6 enforcement: `can()` helper gets a new `canCopy` mode that allows read + clone but not write.

12.3 Cross-chapter events — all members see all events, default-filter to own chapter

Decision. All members can see all events across all chapters globally (cross-chapter and intra-chapter). The default view auto-filters to the user's own chapter. A "See all global events" filter button (toggle) lets the user expand the view to every event in every country.

Rationale. Members travel. A Tel Aviv member visiting Montreal should be able to see Montreal events without creating a Montreal account. Defaulting to own chapter prevents noise; the toggle gives one-click access to the global event calendar.

Implementation impact. Phase 2 public routes: `src/app/events/page.tsx` reads `User.chapterId` from session, default `WHERE chapterId = user.chapterId`. New query param `?scope=all` removes the filter. UI gets a toggle button in the events list header. Phase 6 enforcement: `scopeEventWhere` is updated to support a `scope: "all"` mode that returns cross-chapter rows for read endpoints.

12.4 Country-tier email templates — chapter admins can change only own, copy others

Decision. A chapter admin (CHAPTER_ORGANIZER) and chapter admin can edit only their own chapter's email templates. They cannot edit other chapters' templates. They CAN browse other chapters' templates (read-only) and save-as-copy to use as their own (per decision 12.2). Country-tier overrides are supported: a Country Admin can set a country-level template that all chapters in that country inherit from (chapter-NULL → country-NULL → global-default resolution).

Rationale. This mirrors decision 12.2's pattern at the email-template tier. Country-level templates enable language defaults (Hebrew for Israel, French for Canada) without forcing every chapter to set their own.

Implementation impact. Phase 4 email tiering: `TierEmailTemplateOverride` table supports all three tiers (`tier: "global" | "country" | "chapter"` with corresponding nullable `countryId` / `chapterId`). Resolution order: chapter → country → global. Phase 6 enforcement: `canActOnChapter` extended to template mutations. New copy API endpoint shared with 12.2.

12.5 Super Admin allowlist — stay code-driven for security

Decision. `SUPER_ADMIN_EMAILS` stays hard-coded in `src/lib/permissions.ts` (currently `{"eze@massapro.com"}`). Adding a new Super Admin requires a code change + redeploy. No `User.isSuperAdmin` DB column, no admin UI for granting Super Admin.

Rationale. Super Admin is the highest-privilege role (global read/write across all countries and chapters). Making it DB-driven would mean a single compromised ADMIN account could escalate itself to SUPER_ADMIN. Code-driven means an attacker would need to compromise the source repository + CI/CD pipeline — a meaningfully higher bar.

Implementation impact. No schema change. Document the workflow in `docs/runbooks/secrets-rotation.md`: adding a Super Admin = open PR → code review by existing Super Admin → merge → deploy. Phase 6 enforcement unchanged.

12.6 i18n scope — English-only at V7.1, translations in a follow-up

Decision. Ship Phase 5 with en-US only. Defer he-IL (Israel) and fr-CA (Canada) translations to a V7.2 follow-up release. The timezone migration (Phase 5a + 5b) still ships in V7.1 — only the UI string translation is deferred.

Rationale. Phase 5 is already the largest phase by line count (177 `Asia/Jerusalem` occurrences across 43 files, ~5-10× the original estimate). Bundling full i18n into the same release would make the PR unreviewable and likely ship with regressions. en-US first gives the engineering team a clean burn-in period on the timezone refactor before adding the translation layer.

Implementation impact. Phase 5 scope reduced: no `next-intl` integration, no message catalogs, no locale routing. The `src/lib/datetime-tlv.ts` → `src/lib/chapter-tz.ts` refactor still ships. V7.2 follow-up plan will add `next-intl` + message extraction + he-IL + fr-CA catalogs.

12.7 Member auto-chapter on RSVP — auto-set chapterId if NULL

Decision. When a user RSVPs to an event, auto-set `User.chapterId = event.chapterId` ONLY IF `User.chapterId IS NULL`. If the user already has a `chapterId`, do NOT overwrite it.

Rationale. The "only if NULL" rule preserves the user's primary chapter affinity — a Tel Aviv member who RSVPs to a one-off Montreal event while traveling should NOT have their chapter silently flipped to Montreal (that would break their default events filter per decision 12.3). The first-RSVP backfill is needed because V7 README Q5 left it as a TODO — new users who sign up via `/login?chapterSlug=tel-aviv` already get `chapterId=tel-aviv` at signup, but users who sign up via the generic `/login` form enter the system with NULL `chapterId` and currently stay NULL forever.

Implementation impact. Phase 2 backend: `src/app/api/events/[slug]/rsvp/route.ts` `db.eventRsvp.upsert` create block gets a conditional `UPDATE User SET chapterId = ? WHERE id = ? AND chapterId IS NULL` after the RSVP succeeds. Phase 6 enforcement: with this backfill in place, every RSVP row will have a non-NULL `chapterId` (matching its event's `chapterId`), so chapter organizers' scoped RSVP lists will be populated.

12.8 Media Library — unified `/admin/creatives` with kind filter + new `BrandAsset` table

Decision. Apply both recommendations from the original Section 12 Q8 + Q9: (a) build a unified `/admin/creatives` Media Library with a kind filter (logos / favicons / hero images / banners / mockups / social posts / OG images), and (b) introduce a new `BrandAsset` table to model tier-aware brand-image inheritance (replacing the ad-hoc `SiteSetting`/`ChapterSetting` key-value approach).

Rationale. The existing key-value tables (`SiteSetting`, `ChapterSetting`) don't model tiers cleanly — adding `tier` + `countryId` columns to them would require touching every read site and would still leave the data model ambiguous (a single row could be chapter-scoped or global depending on which FKs are NULL). A dedicated `BrandAsset` table with `(tier, tierId, kind, value, inheritFromParent)` columns is explicit, queryable, and resolves inheritance at read time via a single `resolveBrandAsset(kind, chapterId)` function. The unified Media Library UI gives admins one place to upload and manage every visual asset — separate panels per kind would force 7 different admin pages with 80% duplicate code.

Implementation impact. Phase 3 ships both: `BrandAsset` table + resolver + `/admin/creatives` page + 6 new API routes (CRUD + assign + bulk-tag). `src/lib/chapter-brand-images.ts` is rewritten to read from `BrandAsset` first, gated by `BRAND_ASSET_TABLE_ENABLED` flag for safe rollback. Backfill script `backfill-brand-asset-table.ts` walks `SiteSetting` + `ChapterSetting` + `Chapter.heroImageUrl` and inserts equivalent `BrandAsset` rows.

12.9 Cleanup phase timing — ship Phase 7 immediately after Phase 6

Decision. Ship Phase 7 (drop legacy `Event.chapter String` column + delete dead-code files: `src/lib/v7-scope.ts`, `Event.chapter` readers, unused migrations) immediately after Phase 6 ships to production. No V7.1 burn-in wait.

Rationale. Phase 7 is purely additive cleanup — it removes code that has already been replaced by Phase 1-6 work and drops a column that is no longer read by any code path post-Phase 6. Delaying it would leave dead code in the repository, increasing the surface area for future bugs and making onboarding harder. The risk of a Phase 6 regression that depends on the legacy `Event.chapter` String column is mitigated by the Phase 6 scope-leak integration tests — if any reader of `event.chapter` remains, the test suite will fail before Phase 7 lands.

Implementation impact. Phase 7 sequence: (1) grep audit for `\.chapter` on Event instances (must return 0 results outside the legacy migration files); (2) drop migration `ALTER TABLE "Event" DROP COLUMN "chapter"`; (3) delete `src/lib/v7-scope.ts`; (4) delete `prisma/migrations/V7-add-hierarchy/` (superseded by `20260719000000_v7_add_hierarchy/`); (5) deploy. If the grep audit returns ANY non-migration hits, Phase 7 is blocked until those readers are migrated to `event.chapterRef.name`.

Decisions summary table.

#	Decision	Phase impact	Risk
1	Flat event URLs + city-root aliases + country landing pages	Phase 2	Low — additive routes only
2	Chapter admin: read-only browse + copy other chapters	Phase 2, 6	Low — `canCopy` mode added
3	All events visible globally, default-filter to own chapter	Phase 2, 6	Medium — toggle UI + scope refactor
4	Country-tier email templates + chapter copy	Phase 4, 6	Medium — new tier resolution path
5	Super Admin stays code-driven	`None`	None — current state preserved
6	English-only at V7.1, defer he-IL + fr-CA to V7.2	Phase 5 reduced	Low — i18n deferred, timezone still ships
7	RSVP auto-sets chapterId only if NULL	Phase 2	Low — single conditional UPDATE
8	Unified Media Library + new BrandAsset table	Phase 3	Medium — backfill + resolver rewrite
9	Phase 7 ships immediately after Phase 6	Phase 7	Low — additive cleanup, gated by grep audit

---

END OF PLAN.

Implementation Feasibility Addendum (IMPL-1)

Stress-test of PLAN-1 against the actual V7 codebase. Confirms, refines, or corrects every section of the plan based on a line-by-line audit of `prisma/schema.prisma`, `src/lib/permissions.ts`, `src/lib/auth-guards.ts`, `src/lib/email-campaign/sender.ts`, `src/lib/email-orchestrator/worker.ts`, and 30+ other source files.

This addendum stress-tests PLAN-1's 7-phase migration against the actual source code. Each verdict is GREEN (ship as planned), YELLOW (ship with caveats), or RED (rework needed before shipping). Section G proposes a revised sequence; Section H estimates effort and lists the top 5 risk hotspots.

A. Per-Phase Feasibility Verdict

Phase 1 — Schema additive migration + backfill

Verdict: YELLOW.

Shippable in principle (additive ALTER TABLEs + idempotent backfills), but three hidden dependencies must be addressed first:

1. **Build-script footgun.** `package.json` line 7 runs `prisma migrate deploy 2>&1 || prisma db push --accept-data-loss 2>&1` as part of the Vercel build. If the new migration has ANY error (typo, missing FK, partial apply), the build silently falls through to `db push --accept-data-loss` against production Neon. This can drop columns or reset sequences. This must be fixed BEFORE Phase 1 — change the build script to `prisma migrate deploy` only (no fallback), and let the build fail loudly on migration errors.

2. **EmailQueue / EventRsvp / Speaker write-path is NOT fixed.** PLAN-1 lists these under section 1.1 "Models that ALREADY have chapterId — verify scope, add country inheritance" with "Migration: None." This is misleading. The schema has the column, but the code paths that CREATE rows do not write it:

- `src/lib/email-orchestrator/worker.ts:114` — `db.emailQueue.create` for stage 1 bootstrap (no chapterId)
- `src/lib/email-orchestrator/worker.ts:226` — `db.emailQueue.create` for next-stage rows (no chapterId)
- `src/lib/email-orchestrator/worker.ts:438` — `db.emailQueue.create` for alt-resend rows (no chapterId)
- `src/lib/email-orchestrator/flow-trigger.ts:139, 205, 314` — three `db.emailQueue.create` call sites (no chapterId)
- `src/app/api/events/[slug]/rsvp/route.ts:104` — `db.eventRsvp.upsert create` (no chapterId; V7 README Q5 TODO)

The Phase 2 backfill script will report "0 NULL rows" immediately after running, but NEW rows created by these code paths will keep arriving with NULL chapterId until the code is fixed. This work is currently distributed across Phase 4 (email) and Phase 6 (enforcement) in PLAN-1 — it should be consolidated INTO Phase 1 so the backfill's "0 NULLs" claim is durable.

3. **Phase 1 + Phase 4 are NOT separable on Vercel.** The build script runs `prisma migrate deploy` THEN `next build` in the same Vercel build step. There is no way to "run the migration" without also deploying the code that uses it. PLAN-1's Phase 1 (schema) and Phase 4 (code deploy) are presented as separate steps — on Vercel they are one atomic deploy. The migration SQL and the Prisma client regeneration must be in the same commit, and the code in that commit must not yet READ the new columns (or must handle them gracefully if NULL).

Hidden dependencies:

- `package.json:7` — build script `db push --accept-data-loss` fallback (catastrophic risk)

- `src/lib/email-orchestrator/{worker,flow-trigger}.ts` — 6 `db.emailQueue.create` sites missing chapterId
- `src/app/api/events/[slug]/rsvp/route.ts:104` — EventRsvp create missing chapterId
- `prisma/migrations/20260719000000_v7_add_hierarchy/migration.sql` — already applied; new migration must chain cleanly

Recommended sequence adjustment: Promote the write-path fixes (EmailQueue + EventRsvp + Speaker chapterId) from "Phase 4/6 verification" INTO Phase 1. Add a Phase 0 that fixes the build script and the `/api/admin/members` GET scope bug (1-2 days, ships immediately as a hotfix).

Phase 2 — Scope switcher UI + `withScope()` wrapper

Verdict: YELLOW.

Shippable as a feature-flagged UI addition, but four gotchas need addressing:

1. **No `src/app/admin/layout.tsx` exists.** PLAN-1's section 2.4 says "Add a persistent scope switcher in `AppHeader`." `AppHeader` is an async Server Component (`src/components/ais/app-header.tsx:21`) used on EVERY page (public + admin), not just admin. Adding the switcher to AppHeader means it renders for members too (hidden via client-side check, but the SSR still executes). RECOMMEND: create a new `src/app/admin/layout.tsx` Server Component that wraps the admin subtree and renders `<AppHeader />` + `<ScopeSwitcher />` + `<AdminTabs />` once. This requires moving the per-page `<AppHeader />` + `<AdminTabs />` calls OUT of the 16 admin page.tsx files — a mechanical but wide-reaching refactor.

2. **`getCurrentUser()` signature change ripples through 30+ routes.** PLAN-1 says "Update `getCurrentUser()` to read the scope cookie." But `getCurrentUser()` returns `{user, error, scope}` where `scope = await getUserScope(user.id)` (no cookie). Changing this to `scope = await getEffectiveScope(user.id, cookie)` changes the scope returned for EVERY route that uses `getCurrentUser()`, not just the ones that opt in. During the rollout window, some routes will see the narrowed scope (via `withScope()`) and others won't (via `getCurrentUser()` directly). This is acceptable IF the scope switcher defaults to "natural scope" (no cookie) — but the moment a Super Admin sets a scope override, the inconsistency begins. RECOMMEND: `getCurrentUser()` reads the cookie too (so all routes see the same effective scope), and `withScope()` is a thin wrapper that adds the `scope.kind === "none"` → `403` rejection.

3. **Next.js 16 route handler `params` are now `Promise<{...}>`.** PLAN-1's `withScope()` wrapper signature is `params: Record<string, string>` — this is the Next.js 14/15 shape. In Next.js 16, dynamic route handlers receive `params: Promise<{ key: string }>`. The wrapper must be:

``ts type ScopedHandler = (req: NextRequest, ctx: { params: Promise<Record<string,string>>, user: User, scope: UserScope }) => Promise<NextResponse>; export function withScope(handler: ScopedHandler) { return async (req: NextRequest, ctx: { params: Promise<Record<string,string>> }) => { const { user, error } = await getCurrentUser(); if (error || !user) return NextResponse.json({ error: "Unauthorized" }, { status: 401 }); const scope = await getEffectiveScope(user.id, req.cookies.get("scope")?.value); if (scope.kind === "none") return NextResponse.json({ error: "Forbidden" }, { status: 403 }); const params = await ctx.params; return handler(req, { params, user, scope }); }; } `` This is a Next.js 16-specific gotcha PLAN-1 missed.

4. **Cookie validation defense-in-depth.** `POST /api/admin/scope` validates the override is narrower than the user's natural scope. But `getEffectiveScope(userId, cookie)` must ALSO validate (in case the cookie was tampered with or the user's role was demoted after the cookie was set). A Country Admin whose role was demoted to MEMBER still has the `scope=country:IL` cookie — `getEffectiveScope` must re-check the user's current role and reject the override if it's now wider than their natural scope.

Hidden dependencies:

- `src/components/ais/app-header.tsx` — Server Component; switcher placement requires either admin layout or AppHeader refactor
- `src/lib/auth-guards.ts:47` — `getCurrentUser()` doesn't read cookies; signature change ripples
- `src/lib/auth.ts:209-249` — JWT callback doesn't store chapterId/countryId; scope is re-fetched per request (OK for cookie-based override)
- Next.js 16 `params: Promise<...>` change in route handlers

Phase 3 — Brand asset + Creative panels

Verdict: GREEN.

The cleanest phase. The `BrandAsset` table design (tier enum + countryId + chapterId + kind + inheritFromParent) is sound. The `Creative` table is a straightforward asset registry. The `resolveBrandAsset(kind, chapterId, countryId)` resolver is a 3-step walk that's easy to test.

One addition needed: `revalidateTag` **after BrandAsset mutations.** PLAN-1 section 6.4 mentions "in-memory cache with 60-second TTL" but doesn't address Next.js's RSC fetch cache. Every public page that resolves branding (`/c/[chapterSlug]`, `/login?chapterSlug=`, `/e/[slug]`, `/[chapterSlug]`) caches the `generateMetadata()` result. When a Super Admin uploads a new chapter logo, the change must propagate immediately. Use `revalidateTag(brand-${chapterId}-${kind})` in every BrandAsset POST/PATCH/DELETE handler. PLAN-1 doesn't mention `revalidateTag` anywhere — gap.

Hidden dependencies:

- `src/lib/chapter-brand-images.ts` — current resolver only handles 3 keys (favicon, loginHero, loginBanner); rewrite must preserve back-compat
- `src/app/admin/images/page.tsx` — current conflated panel; must remain working until callers migrated
- `src/app/login/page.tsx`, `src/app/c/[chapterSlug]/page.tsx`, `src/app/layout.tsx` generateMetadata — all call `getEffectiveBrandImagesBySlug`; must be updated to call `resolveBrandAsset`

Polymorphic relation recommendation (for Section C): Use `tier` enum + 3 nullable FKs (countryId, chapterId), NOT a polymorphic `tierType` + `tierId` string. The 3-FK approach preserves Prisma's typed relations + Postgres FK constraints. The `@@unique([tier, countryId, chapterId, kind])` constraint has a subtle NULL-distinct issue in Postgres (multiple `tier=country, countryId=NULL` rows could coexist) — mitigate with app-level validation in the POST handler (reject if `tier=country` and `countryId` is null).

Phase 4 — Email system completion

Verdict: YELLOW.

Shippable but three issues:

1. **PLAN-1's file list is inaccurate.** Section 5.4 says "replace hard-coded `fromName = "AI Salon Tel Aviv"` in `src/lib/email-campaign/sender.ts`." But `sender.ts:92` actually uses `campaign.fromName || "AI Salon"` (NOT "Tel Aviv"). The hard-coded "AI Salon Tel Aviv" strings are in:

- `src/app/api/cron/email/route.ts:97,175`
- `src/app/api/admin/email/campaigns/[id]/send/route.ts:79`
- `src/app/api/email/unsubscribe/route.ts:58,60`
- `src/app/api/speakers/[id]/messages/route.ts:153`
- `src/app/api/messages/[userId]/route.ts:189,195,199,209`

PLAN-1's section 5.4 lists these but misattributes the sender.ts fallback. The Phase 4 implementer must update BOTH the route handlers AND the sender library — the sender uses `SMTP_USER` for fromEmail (not `ADMIN_EMAIL`), so the env-var fallback chain is different than the plan describes.

2. `sender.ts` **does NOT write chapterId to EmailEvent rows.** Lines 153-160 and 170-178 create `db.emailEvent.create({ data: { campaignId, recipientId, email, type } })` — no chapterId. Phase 6 enforcement on EmailEvent (which Phase 1 adds chapterId to) will hide these events from chapter organizers' analytics dashboards. Phase 4 must add `chapterId: campaign.chapterId` to every `db.emailEvent.create` call (2 sites in sender.ts + 2 in worker.ts).

3. **Vercel function timeout + IMAP multi-inbox.** The orchestrator worker (`/api/cron/email`) processes 200 PENDING EmailQueue rows per run. Adding `resolveStageTemplate(stageTemplateId, chapterId, countryId)` per row (3 DB queries each) pushes per-row time from ~150ms to ~400ms → 200 rows × 400ms = 80s, exceeding the 60s Vercel function timeout. Mitigation: cache tier resolution by `(stageTemplateId, chapterId, countryId)` for the duration of one worker run using a `Map<string, ResolvedTemplate>`. PLAN-1 doesn't mention this.

For IMAP polling (`/api/cron/email/imap-poll`): currently polls ONE inbox. Phase 4 extends to N inboxes (one per chapter's replyTo). Polling 5+ inboxes sequentially will timeout. Mitigation: poll one inbox per cron tick (round-robin via a `lastPolledInbox` cursor in SiteSetting) OR deploy a separate Vercel Cron per inbox.

Hidden dependencies:

- `src/lib/email-campaign/sender.ts:92,153,170` — fromName fallback + EmailEvent create without chapterId
- `src/lib/email-orchestrator/worker.ts:114,226,438` — EmailQueue create without chapterId (Phase 1 fix)
- `src/app/api/cron/email/imap-poll/route.ts` — single-inbox polling; multi-inbox needs round-robin
- `src/lib/relay-recipients.ts` — DRAFT; must be wired into 2 routes (speaker messages + DMs)

Phase 5 — Timezone + UI copy de-hardcoding

Verdict: RED.

This is the most underestimated phase. PLAN-1 claims "15 file:line refs" for `Asia/Jerusalem`. The actual count is **177 occurrences across 43 files** (verified via grep). The plan's effort estimate is off by 5-10×.

Breakdown of the 177 occurrences:

- `src/lib/datetime-tlv.ts` — 15 occurrences (the central helper module; every importer inherits the hard-coding)
- `src/app/admin/check-in/door-check-in-client.tsx` — 10 (CLIENT component; needs `tz` prop)
- `src/app/admin/events/admin-events-list-with-actions.tsx` — 4
- `src/app/events/events-list.tsx` — 8 (CLIENT component)
- `src/app/events/[slug]/page.tsx` — 6
- `src/app/events/[slug]/tabs/admin-agenda-tab.tsx` — 9 (CLIENT component)
- `src/app/admin/mockups/speaker-intro/event-mapper.ts` — 6
- `src/app/admin/mockups/shared/time-format.ts` — 6 (shared helper)
- `src/app/e/[slug]/public-event-page.tsx` — 6
- `src/components/admin/event-editor.tsx` — 8
- Plus 32 more files with 1-5 occurrences each

Three structural problems PLAN-1 doesn't address:

1. **Most occurrences are in CLIENT components.** Client components can't call `getChapterTimezone(chapterId)` (async DB lookup). The timezone string must be passed from the server component as a prop. This requires refactoring the data flow: server component fetches chapter → passes `tz` string to every child client component. Many of these children are 3-5 levels deep in the component tree — prop drilling is non-trivial. Alternative: React Context (`ChapterTimezoneProvider`), but Context doesn't cross the RSC/Client boundary by default — the value must be serialized in the server component and passed to a Client Context.Provider.

2. `src/lib/datetime-tlv.ts` **is a helper module named "tlv" that hard-codes Asia/Jerusalem.** Every file that imports from it inherits the hard-coding. The fix is to either (a) rename to `datetime.ts` and make all functions take a `tz` parameter, OR (b) create a new `chapter-datetime.ts` and migrate callers incrementally. Option (a) is cleaner but touches every importer; option (b) is safer but creates a transitional period with two helper modules.

3. **i18n wiring is bundled into Phase 5 but shouldn't be.** PLAN-1 section 7.5 says "Ship Phase 5 with UI i18n for en-US only. Add he-IL + fr-CA in a follow-up." But en-US is ALREADY the implicit default (all UI strings are English). Wiring `next-intl` for en-US only adds complexity (NextIntlClientProvider, messages/en-US.json, useTranslations calls) without value. RECOMMEND: defer ALL i18n wiring to Phase 7, and keep Phase 5 focused on timezone + UI copy de-hardcoding only.

Recommended split:

- **Phase 5a (3-5 days):** Create `src/lib/chapter-tz.ts` with `getChapterTimezone()` + `formatInTz()`. Migrate `src/lib/datetime-tlv.ts` to delegate to it. Update all SERVER components to pass `tz` as props.
- **Phase 5b (5-7 days):** Migrate CLIENT components to accept `tz` prop (or use a ChapterTimezoneProvider). Replace hard-coded "AI Salon Tel Aviv" / "Tel Aviv Chapter" strings with chapter-aware helpers.
- **Phase 5c (DEFER to Phase 7):** Wire next-intl.

Hidden dependencies:

- `src/lib/datetime-tlv.ts` — 15 occurrences; central helper that must be refactored first

- 30+ client components that need `tz` prop drilling or Context
- `src/app/layout.tsx:53,54,57-65,68,84` — metadata title template; needs `generateMetadata()` per route (not just layout-level)
- `src/components/ais/app-header.tsx:71` — "Tel Aviv Chapter" badge; needs chapter context from route

Phase 6 — Scope enforcement (read + write paths)

Verdict: YELLOW.

Shippable but the migration scope is bigger than PLAN-1 acknowledges.

1. **Only 2 admin API routes use scope helpers today.** Verified via grep: `src/app/api/admin/events/route.ts` (POST only, uses getUserScope for chapterId validation) and `src/app/api/admin/analytics/route.ts`. The other ~30 admin API routes do NOT scope. PLAN-1's section 8.1 table claims 12 routes "already use scopeWhere()" — this is INACCURATE. The 12 routes listed are admin PAGES (which scope via SSR), not API routes. The Phase 6 work to wrap "every admin list endpoint in `withScope()`" is actually ~30 routes, not ~12.
2. `/api/admin/members` **GET scope fix is a critical security fix that should NOT wait for Phase 6.** It's a 1-line change (`const where = { archivedAt: null, ...scopeUserWhere(scope) }`) that closes a real data leak TODAY (any Admin can see ALL members globally). This should be Phase 0, not Phase 6.
3. **Canary strategy missing.** PLAN-1 says "Flip `V7_TIER_ENFORCEMENT=on`. Possible brief spike in 403s if any route is missing the scope helper — monitor Sentry." This is risky. RECOMMEND: ship `withScope()` as a LOGGING-ONLY wrapper for 1 week (logs scope mismatches but doesn't reject), then flip to enforcing. This catches missing scope helpers without breaking production.
4. **Socket.IO mini-services need scope enforcement.** `mini-services/chat-service/index.ts` is a stateless relay — it trusts the client on `chat:room:join` (line 162). A malicious user could join any room's socket and receive broadcasts. Phase 6 should add a server-side check: the chat service calls back to a Next.js endpoint (`/api/chat/rooms/[roomId]/verify-membership?userId=X`) before allowing `chat:room:join`. Same for quiz-service. This is a non-trivial change to the mini-service architecture that PLAN-1 doesn't mention.
5. **RSVP write-path must be fixed BEFORE Phase 6.** As noted in Phase 1: `src/app/api/events/[slug]/rsvp/route.ts:104` creates EventRsvp without chapterId. Phase 6 enforcement on EventRsvp will exclude these from chapter organizer views (the organizer sees an empty registrants list for their own event). Must fix in Phase 1, not Phase 6.

Hidden dependencies:

- ~30 admin API routes that use `getCurrentUser()` directly (must migrate to `withScope()`)
- `mini-services/chat-service/index.ts:162` — `chat:room:join` trusts client
- `mini-services/quiz-service/index.ts` — same pattern
- `src/app/api/events/[slug]/rsvp/route.ts:104` — EventRsvp create without chapterId

Phase 7 — Cleanup + URL aliases + i18n

Verdict: YELLOW.

Three risks:

1. `Event.chapter String` **column drop is dangerous.** PLAN-1 says "all readers migrated to `chapterRef.name` in Phase 5." But Phase 5 (even done correctly) only migrates the TIMEZONE + UI COPY references. A grep for `event.chapter` (the String property, NOT the relation `event.chapterRef`) is needed to find every reader. There are likely 5-10 files that read `event.chapter` as a display string (event cards, email subjects, etc.). If ANY reader remains when the column is dropped, it throws at runtime. The build script's `db push --accept-data-loss` fallback means the drop could succeed even if the migration is malformed. RECOMMEND: before Phase 7, run a grep audit for `\.chapter\b` (word boundary) on Event instances, migrate every reader, then drop.
2. **City-root alias `/[chapterSlug]` causes 404 UX regression.** `src/app/[chapterSlug]/page.tsx` is a catch-all dynamic segment. Next.js prioritizes static segments (`/login`, `/events`, `/admin`) over dynamic ones, so those win. But `/<unknown-typo>` (e.g. `/evnts`) would match `[chapterSlug]` and `notFound()` after a DB lookup — instead of the normal Next.js 404 page. This adds a DB query to every 404. Mitigation: cache the list of valid chapter slugs in-memory (refreshed every 60s) and `notFound()` without a DB query if the slug isn't in the cache.
3. **i18n (he-IL + fr-CA) is a multi-week effort.** Requires translators, RTL support for he-IL (CSS `dir="rtl"`, Tailwind logical properties), locale-aware date/number formatting, and a `getRequestConfig` middleware. PLAN-1 bundles this into Phase 7 alongside cleanup — it should be a separate Phase 8.

Hidden dependencies:

- Every file that reads `event.chapter` (String property) — must be migrated before column drop
- `src/app/[chapterSlug]/page.tsx` — new catch-all route; 404 UX regression
- `src/app/[countrySlug]/page.tsx` — new country landing page
- `messages/he-IL.json`, `messages/fr-CA.json` — translation files (multi-week effort)
- `src/lib/v7-scope.ts` — delete (dead code)
- `src/lib/admin-auth.ts` — delete after migrating all callers

B. Next.js 16 App Router Specifics

B.1 Server vs Client component classification for new admin pages

Page	Recommendation	Why
`/admin/branding/global`	**RSC** (Server Component)	Reads BrandAsset rows server-side; no client interactivity until upload modal opens
`/admin/branding/countries/[id]`	**RSC**	Same; country id from params
`/admin/branding/chapters/[id]`	**RSC**	`Same`
`/admin/creatives`	**RSC shell + Client grid**	The grid (filter bar, drag-drop upload, detail drawer) must be Client; the page shell (auth check, initial query) is RSC
`/admin/email/templates/overrides`	**RSC shell + Client matrix**	The 3-column matrix editor (per-cell rich text via @mdxeditor) must be Client; the shell is RSC
`<ScopeSwitcher />`	**Client** (inside RSC AppHeader or admin layout)	Dropdown interactivity, cookie POST on change
`/admin/scope API`	Route Handler	`Standard`

B.2 Where `currentScope` state should live

Recommendation: HttpOnly cookie named `ais_scope`, 30-day expiry, SameSite=Lax.

Justification:

- **URL search param:** Pollutes every link; lost on navigation; bad UX.
- **Server session (DB column on User):** Persists across logouts (bad — a Super Admin who switched to "Israel" and forgot to reset would be confused on next login from a different device). Also requires a DB write on every scope change.
- **JWT claim:** next-auth v4 JWT is signed; changing it requires a session update (re-issuance). The current JWT callback (`src/lib/auth.ts:209`) re-fetches role from DB on every request — adding `scopeOverride` to the JWT would require updating the callback + the session callback. Doable but couples scope to the auth session.
- **HttpOnly cookie:** Decouples scope from auth. Survives page refresh. Cleared on logout (if `SameSite=Lax` + cookie path=/). Can be read by RSC via `cookies()` from `next/headers`. Can be set by a Route Handler via `NextResponse.cookies.set()`. Validated server-side on every request by `getEffectiveScope()`.

The cookie value should be a signed string like `global` | `country:IL` | `chapter:tel-aviv-id` — NOT a JSON blob (simpler to validate, no parsing errors).

B.3 Scope switcher interaction with `cookies()` / `headers()` in RSC

In a Server Component, read the cookie via:

```
import { cookies } from "next/headers";
async function AdminLayout({ children }) {
  const scopeCookie = (await cookies()).get("ais_scope")?.value;
  const scope = await getEffectiveScopeFromCookie(scopeCookie);
  return <ScopeSwitcherClient currentScope={scope} />;
}
```

Note: in Next.js 16, `cookies()` returns a `Promise<ReadonlyRequestCookies>` — it must be awaited. PLAN-1's code samples don't await `cookies()`, which would fail in Next.js 16.

B.4 Parallel Routes vs Intercepting Routes for the tier-selector drawer

Recommendation: neither. Use a plain Radix Dialog (shadcn/ui `<Sheet />` or `<Dialog />`).

- **Parallel Routes (`@tier-selector`):** Overkill for a single dropdown. Parallel Routes are designed for dashboard-style layouts where multiple panels coexist permanently. The scope switcher is a transient dropdown.
- **Intercepting Routes (`(..)tier-selector`):** Designed for modal galleries (click a photo → modal opens with the photo, but the URL changes so refresh works). The scope switcher doesn't need a URL — it's a cookie. Intercepting Routes add URL complexity for no benefit.
- **Plain Dialog:** The `<ScopeSwitcher />` client component renders a `<Sheet>` (vaul drawer) or `<Dialog>` (Radix) with the country/chapter tree. On select, POSTs to `/api/admin/scope` which sets the cookie + returns the new scope. The client then calls `router.refresh()` to re-render the page with the new scope. Simple, no URL pollution.

B.5 `revalidatePath` / `revalidateTag` after scope-scoped mutations

`revalidateTag` **is required after BrandAsset mutations.** Tags:

- `brand-${chapterId}-${kind}` — invalidate when a BrandAsset row for this chapter+kind changes
- `brand-${countryId}-${kind}` — invalidate when a country-tier BrandAsset changes
- `brand-global-${kind}` — invalidate when a global BrandAsset changes
- `chapter-landing-${chapterId}` — invalidate `/c/[chapterSlug]` page cache
- `chapter-login-${chapterId}` — invalidate `/login?chapterSlug=` metadata cache

`revalidatePath` **is required after scope-scoped list mutations.** When a Super Admin (switched to chapter scope) creates an event, revalidate:

- `/admin/events` (the admin list)
- `/c/[chapterSlug]` (public chapter landing, shows upcoming events)
- `/events` (public events list)

Call `revalidatePath` in the POST handler AFTER the db.create succeeds.

PLAN-1 mentions neither `revalidatePath` nor `revalidateTag` — significant gap.

C. Prisma 6 Specifics

C.1 Transaction strategy for backfills

Neon connection pool: The user's Neon plan is NOT visible in the repo (no `vercel.json` Neon config, no `.neon` file). The schema comment says "Vercel production (Vercel Postgres / Neon)." Neon Free tier = 5 pool connections; Pro = 20; Scale = 100. The backfill scripts run server-side via `npx tsx scripts/...` (NOT as Vercel functions), so they don't compete with the Vercel function pool — they use their own connection. But if the backfill runs WHILE the app is serving traffic, both share the same Neon pool. RECOMMEND: run backfills during low-traffic hours (Sunday 02:00 UTC) and use `--batch-size=500 --sleep=200` to stay under 5 concurrent connections.

Prisma transaction limits: Prisma 6's `$transaction` has a default timeout of 5s (configurable up to 60s on Pro, longer on self-hosted). The Phase 2 backfill of EmailEvent (~50K rows) + EmailRecipient (~50K rows) via a single `UPDATE` would exceed this. PLAN-1's "batches of 1000 rows with 100ms sleep" is correct — each batch is a separate statement (not a transaction), so no timeout issue. But the backfill script must NOT wrap all batches in a single `$transaction` — that would timeout. Use individual `db.$executeRaw` calls per batch.

C.2 Prisma 6 client extensions for `scoped()` query modifier

Recommendation: NO. Stick with PLAN-1's `withScope()` wrapper.

Rationale:

- Prisma 6's `$extends()` is applied globally to the `db` client. A multi-tenant scope would need `AsyncLocalStorage` to pass the per-request scope through, which is fragile in Vercel's serverless environment (each function invocation is isolated, but AsyncLocalStorage doesn't always propagate correctly across awaited boundaries in Next.js route handlers).
- Extensions can't easily express the conditional logic (country scope uses `chapter.countryId` join; chapter scope uses `chapterId` directly; global scope uses no filter). The `scopeUserWhere(scope)` helper functions already do this cleanly.
- Explicit `withScope()` wrapper makes the scope VISIBLE in every route handler — easier to audit, easier to test. A `db.user.scoped().findMany()` extension hides the scope, making it easy to forget.
- The `withScope()` wrapper also handles auth (401 if not signed in) + scope rejection (403 if scope is "none") in one place. An extension would only handle the query filter, not the auth.

C.3 Index strategy for new `chapterId` / `countryId` columns

Composite index on `(chapterId, createdAt)` for time-ordered queries. Most admin list pages filter by chapter + sort by createdAt. A bare `@@index([chapterId])` forces a sort after the filter. Add `@@index([chapterId, createdAt])` on: EmailQueue, EmailRecipient, EmailEvent, TrackingLog, Testimonial, ChatMessage, EventImage.

Partial index on `chapterId WHERE chapterId IS NOT NULL` for backfill verification. After Phase 2 backfill, the "0 NULL rows" check needs a fast count. A partial index `CREATE INDEX ... ON "EmailEvent"("chapterId") WHERE "chapterId" IS NOT NULL` makes the count fast. But Postgres partial indexes aren't supported by Prisma's schema syntax — they must be added via raw SQL in the migration.

Unique constraint on `BrandAsset`: `@@unique([tier, countryId, chapterId, kind])` is correct. Note Postgres treats NULL as distinct — multiple `tier=country, countryId=NULL` rows could coexist. Add app-level validation in the POST handler (reject if tier=country and countryId is null).

C.4 Polymorphic relation for BrandAsset / Creative

Recommendation: `tier` enum (String) + 3 nullable FKs (countryId, chapterId), NOT polymorphic `tierType` + `tierId`.

Justification:

- **Polorphic (tierType + tierId):** No FK constraint — a BrandAsset could point to a deleted Chapter with no DB-level error. Prisma can't type the relation (would be `tierRef: Chapter? | Country?` which Prisma doesn't support). Hard to query ("get all BrandAssets for chapter X" requires `WHERE tierType='chapter' AND tierId=X` — no index on the polymorphic pair without a composite index).

- **3 nullable FKs:** Prisma types each relation cleanly (`chapter Chapter?`, `country Country?`). Postgres enforces FK constraints (deleting a Chapter cascades or restricts correctly). Querying is natural (`WHERE chapterId = X`). The `tier` String column is app-level metadata that tells the resolver which FK to use. The `@@unique([tier, countryId, chapterId, kind])` constraint prevents duplicates.

For `Creative`, same approach: `tier` + `countryId?` + `chapterId?` + optional `eventId?` (for event-bound creatives).

D. Vercel & Vercel Blob Specifics

D.1 Tier-scoped upload: separate route per tier, or one route with `?tier=`?

Recommendation: one route with `?tier=` query param.

- **Separate routes (`/api/admin/branding/global/upload`, `/api/admin/branding/countries/[id]/upload`, `/api/admin/branding/chapters/[id]/upload`):** More route files, more boilerplate, but clearer URL semantics.
- **One route (`/api/admin/branding/upload?tier=global&kind=logo`):** Less boilerplate, single validation path, easier to add new tiers. The `tier` + `countryId` + `chapterId` come from the query string + body; the route validates the caller's scope covers the requested tier.

Use one route. The validation logic is identical regardless of tier — no benefit to splitting.

D.2 Blob path collision risk

Naming convention: `brand-assets/<tier>/<tierId>/<kind>/<timestamp>-<random>.<ext>`

Examples:

- `brand-assets/global/-/logo/1700000000000-abc123.png`
- `brand-assets/countries/<countryId>/favicon/1700000000000-def456.ico`
- `brand-assets/chapters/<chapterId>/banner/1700000000000-ghi789.jpg`
- `creatives/<tier>/<tierId>/<timestamp>-<random>.<ext>`

The `<timestamp>-<random>` suffix (via `uniqueBlobFilename()` from `src/lib/blob-paths.ts:157`) guarantees uniqueness even if two chapters upload a file with the same original name. The `<tier>/<tierId>/<kind>` prefix makes it easy to list/delete all blobs for a chapter (`list` with prefix `brand-assets/chapters/<chapterId>/`).

Collision risk: ZERO with the timestamp+random suffix. The existing flat `brand-assets/<filename>` convention (used by `/api/admin/brand-images`) can coexist — new uploads use the tier-prefixed convention, old URLs continue to resolve, the `BrandAsset` table maps (tier, tierId, kind) → blobUrl regardless of path.

D.3 Vercel function timeout

- **Default (Hobby):** 10s
- **Pro:** 60s
- **Enterprise:** 300s

At-risk operations:

1. **Email orchestrator worker** (`/api/cron/email`)**:** Processes 200 PENDING rows per run. Current per-row time ~150ms → 30s total. With Phase 4 tier resolution (3 DB queries per row) → ~400ms per row → 80s total. EXCEEDS 60s Pro timeout. Mitigation: cache tier resolution in a `Map` for the duration of one worker run.
2. **Phase 2 backfill** (`scripts/backfill-tier-chapter-ids.ts`)**:** Runs via `npx tsx` server-side, NOT as a Vercel function. No timeout — but Neon's `statement_timeout` (default 15s on pool connections) applies per-statement. Use batches of 500-1000 rows.
3. **Bulk member import** (`/api/admin/members/bulk-import`)**:** xlsx parsing + N User creates. Current limit ~100 rows per 60s function. Tier resolution adds ~0ms (the import is one chapter; resolve once). No new risk.
4. **Image rotation** (`/api/images/rotate`)**:** sharp processing + Vercel Blob put/del. Already near 60s for large images. No new risk from tier scoping.

D.4 Cold-start implications for Socket.IO mini-services

Recommendation: keep `mini-services/chat-service` + `mini-services/quiz-service` as separate deployments (Render/Railway/Fly.io), NOT migrate to Vercel serverless functions.

Rationale:

- **Vercel serverless functions are request-response, not persistent.** Socket.IO needs a long-lived WebSocket connection. Vercel's serverless functions timeout at 60s (Pro) — a Socket.IO connection lasting longer would be killed.
- **Vercel doesn't support sticky sessions.** Socket.IO's in-memory adapter (`socketInfo` Map in `chat-service/index.ts:132`) requires that the same server handles the same client. With serverless, each reconnect may hit a different instance → lost state.
- **Cold-start latency.** Vercel serverless functions cold-start in ~1-3s. A chat message sent to a cold function would be delayed. The current Render/Railway deployment has zero cold-start.

The mini-services should stay as separate Node.js processes. The Caddyfile's `XTransformPort` query param mechanism (port 3003 for quiz, 3004 for chat) already handles routing. Phase 6's scope enforcement should be added as a server-side membership check in the chat-service (call back to `/api/chat/rooms/[roomId]/verify-membership` before allowing `chat:room:join`), NOT by migrating the service to Vercel.

E. Auth & Session Specifics

E.1 next-auth v4 session callback shape

Current JWT contents (`src/lib/auth.ts:209-249`): `{ id, email, role, provider, idResolved }`. NO `chapterId` or `countryId`.

Recommendation: keep chapterId/countryId OFF the JWT. Re-fetch per request.

Justification:

- The JWT callback already does a `db.user.findUnique` on every sign-in + every request where `token.id` isn't resolved. Adding chapterId/countryId to the JWT would save one DB query per request — but

`getCurrentUser()` already does its own `db.user.findUnique({ where: { email } })` (line 56), which fetches chapterId/countryId. So the JWT optimization saves nothing.

- If chapterId/countryId are on the JWT and a Super Admin DEMOTES an Admin to MEMBER, the demoted user's JWT still has `countryId=IL` until they re-auth. Their next request would be scoped to Israel (via the stale JWT) even though they're now a MEMBER with no scope. Re-fetching per request avoids this.
- The scope switcher's cookie override is SEPARATE from the JWT. `getEffectiveScope(userId, cookie)` reads the User row (for natural scope) + the cookie (for override) on every request. Clean separation.

E.2 Scope persistence across logouts

Recommendation: cookie with `SameSite=Lax`, `path=/admin`, 30-day expiry. Does NOT persist across logouts (clear on logout).

- **Persist across logouts (DB column on User):** A Super Admin who switched to "Israel" and forgot to reset would be confused on next login from a different device. Bad UX.
- **Cookie with `SameSite=Lax`, `path=/`, 30-day expiry:** Persists across page refreshes. Survives logout (cookie isn't cleared by next-auth's signOut). BAD — a different user on the same browser would inherit the scope.
- **Cookie with `SameSite=Lax`, `path=/admin`, 30-day expiry, cleared on logout:** Best. The next-auth `signOut` callback should call `cookies().delete('ais_scope')`. Path=`/admin` means the cookie isn't sent on public routes (minor perf win).

E.3 Scope switcher vs `/login?chapterSlug=` branding override

No conflict. They serve different purposes:

- `/login?chapterSlug=tel-aviv` — applies CHAPTER BRANDING (favicon, hero, banner) to the login page. Read by `getEffectiveBrandImagesBySlug(chapterSlug)` in `src/app/login/page.tsx`. Does NOT change the user's admin scope.
- Scope switcher cookie (`ais_scope`) — changes the user's ADMIN scope for `/admin/*` pages. Does NOT change login page branding.

A Super Admin who switched their admin scope to "Montreal" and then visits `/login?chapterSlug=tel-aviv` sees Tel Aviv branding on the login page (correct — they're looking at the Tel Aviv chapter's login) AND Montreal scope in admin (correct — they switched their admin scope). No conflict.

The only edge case: a Super Admin switches scope to "Montreal", then navigates to `/admin/chapters` — the page shows chapters in Montreal's country (Canada). If they then click "Edit" on a Canadian chapter, the chapter editor at `/admin/chapters/[id]` should work (the chapter is in their scope). The scope switcher's job is to NARROW the admin view, not to change branding.

F. Testing & QA Strategy

F.1 Minimal test matrix for scope-leak regressions

Test	Setup	Assertion
Chapter Admin A cannot read Chapter B's emails	Create Chapter A + Chapter B in same country. Create Admin A (chapterId=A). Create EmailCampaign in B.	GET `/api/admin/email/campaigns` as Admin A → response.campaigns does NOT include B's campaign
Chapter Admin A cannot read Chapter B's members	Same setup. Create User in B.	GET `/api/admin/members` as Admin A → response.members does NOT include B's user
Country Admin cannot read other country's events	Create Country X + Country Y. Create Admin X (countryId=X). Create Event in Y.	GET `/admin/events` page render as Admin X → events list does NOT include Y's event
Super Admin switched to Chapter A cannot edit Chapter B's event	Super Admin + scope cookie `chapter:A`. Event in B.	PATCH `/api/admin/events/[B-event-id]` → 403
Member cannot access admin endpoints	Member user.	GET `/api/admin/members` → 403
RSVP to Chapter A event auto-sets User.chapterId=A (after Phase 1 fix)	Member with chapterId=null. Event in A.	POST `/api/events/[slug]/rsvp` → User.chapterId === A.id
Scope switcher cookie tampering rejected	Admin A (countryId=X). Cookie `ais_scope=global`.	`getEffectiveScope(A.id, 'global')` → returns `{kind:'country', countryId:X}` (not global)
BrandAsset inheritance walk	BrandAsset[global, logo] = "logo.png". BrandAsset[country, logo, inheritFromParent=true].	`resolveBrandAsset('logo', chapterInCountry)` → returns "logo.png"

F.2 Vitest vs Playwright

Recommendation: BOTH, scoped as follows.

- **Vitest (unit + integration):** For pure-logic helpers — `getEffectiveScope()`, `scopeUserWhere()`, `resolveBrandAsset()`, `resolveFromEmail()`, `formatInTz()`, `safeBlobPathname()`. Fast, no DB. Mock `db` via `vi.mock('@/lib/db')`.
- **Playwright (E2E):** For the 3 most critical flows: 1. **Scope switcher flow:** Login as Super Admin → switch scope to "Israel → Tel Aviv" → navigate to /admin → verify only Tel Aviv members shown → refresh → scope persists → switch back to "Global" → all members shown. 2. **Email tier resolution flow:** As Super Admin, set `Country[IL].defaultFromName = "AI Salon Israel"` → create campaign in Tel Aviv chapter → send → verify sent email's fromName = "AI Salon Israel" (no chapter override) → set `ChapterSetting[tel-aviv, emailFromName, "AI Salon TLV"]` → send again → verify fromName = "AI Salon TLV". 3. **Cross-chapter isolation flow:** Create Chapter A + Chapter B in same country → create Admin A (chapterId=A) → Admin A logs in → verify /admin/events shows only A's events → verify /admin/email/campaigns shows only A's campaigns → verify direct API call to `/api/admin/events/[B-event-id]` returns 403.

F.3 Scope audit script

Proposed: `scripts/scope-audit.ts` — runs through every admin API endpoint with 3 session tokens (super, country admin, chapter organizer) and asserts no cross-tier data leaks.

```
// Pseudocode (reference snippet only — NOT production code)
const SESSIONS = {
  super: await loginAs("super@test.com"),
  countryAdmin: await loginAs("country-admin@test.com"),  // Admin of Israel
  chapterAdmin: await loginAs("chapter-admin@test.com"),   // Organizer of Tel Aviv
};

const ENDPOINTS = [
  "GET /api/admin/members",
  "GET /api/admin/events",
  "GET /api/admin/registrants",
  "GET /api/admin/speakers",
  "GET /api/admin/email/campaigns",
  "GET /api/admin/email/templates",
  "GET /api/admin/email/flows",
  "GET /api/admin/email/audiences",
  "GET /api/admin/quiz",
  "GET /api/admin/analytics",
  "GET /api/admin/non-members",
  // ... every admin list endpoint
];

for (const endpoint of ENDPOINTS) {
  const [method, path] = endpoint.split(" ");
  for (const [role, session] of Object.entries(SESSIONS)) {
    const res = await fetch(`https://aisalon.massapro.com${path}`, {
      method,
      headers: { cookie: session.cookie },
    });
    const data = await res.json();
    const rows = extractRows(data);
    const leakedRows = rows.filter(r => !isInScope(r, role));
    if (leakedRows.length > 0) {
      console.error(`SCOPE LEAK: ${role} on ${endpoint} saw ${leakedRows.length} rows outsid
e scope`);
      process.exit(1);
    }
  }
}
```

Run this script after Phase 6 deployment + before every subsequent deploy that touches admin API routes.

G. Revised Phase Sequence

PLAN-1's sequence is mostly correct but needs three adjustments:

G.1 Add Phase 0 (hotfixes that should ship immediately)

Phase 0 (1-2 days):

1. Fix `package.json` build script — remove `|| prisma db push --accept-data-loss` fallback. Let build fail loudly on migration errors.
2. Fix `/api/admin/members` GET to scope via `scopeUserWhere(scope)`. (1-line change, closes a real data leak TODAY.)
3. Fix `/admin/testimonials` role gate bug — change `if (me.role !== "ADMIN")` to `if (!can(me.role, "members.view") && !isSuperAdminEmail(me.email))`.

These are critical security/correctness fixes that should NOT wait for Phase 6.

G.2 Consolidate write-path fixes into Phase 1

PLAN-1 distributes the EmailQueue/EventRsvp/Speaker chapterId write-path fixes across Phase 4 (email) and Phase 6 (enforcement). This is wrong — the backfill script's "0 NULL rows" claim is only durable if the write paths are fixed IN THE SAME DEPLOY as the backfill. Move all write-path fixes into Phase 1:

- `src/lib/email-orchestrator/worker.ts` — 3 `db.emailQueue.create` sites
- `src/lib/email-orchestrator/flow-trigger.ts` — 3 `db.emailQueue.create` sites
- `src/app/api/events/[slug]/rsvp/route.ts` — `db.eventRsvp.upsert create`
- `src/app/api/admin/speakers/route.ts` — `db.speaker.create` (verify chapterId from event)
- `src/lib/email-campaign/sender.ts` — 2 `db.emailEvent.create` sites (add chapterId from campaign)

G.3 Split Phase 5 into 5a + 5b; defer i18n to Phase 7

Phase 5 is underestimated 5-10×. Split:

- **Phase 5a (3-5 days):** `chapter-tz.ts` helper + server-side timezone replacement. Migrate `src/lib/datetime-tlv.ts` to delegate to it. Update all SERVER components.
- **Phase 5b (5-7 days):** Migrate CLIENT components to accept `tz` prop. Replace hard-coded "AI Salon Tel Aviv" / "Tel Aviv Chapter" UI strings.
- **Phase 5c (DEFER to Phase 7):** Wire next-intl. Ship he-IL + fr-CA translations.

G.4 Final revised sequence

```
Phase 0 (hotfixes)              — 1-2 days, ships immediately
Phase 1 (schema + backfill + write-path fixes) — 3-5 days
  └ Phase 2 (scope switcher + withScope)       — 3-5 days (parallel with Phase 1 tail)
  └ Phase 3 (BrandAsset + Creative panels)     — 5-7 days (after Phase 1)
  └ Phase 4 (email tier resolver)              — 5-7 days (after Phase 1)
  └ Phase 5a (timezone server-side)            — 3-5 days (after Phase 1)
Phase 5b (timezone client-side + UI copy)      — 5-7 days (after Phase 5a)
Phase 6 (scope enforcement + canary)           — 5-7 days (after Phases 2-5b)
Phase 7 (cleanup + URL aliases + i18n)         — 7-10 days (after Phase 6 burn-in)
```

Total: ~5-7 weeks of engineering effort (1 engineer, sequential). Parallelizable with 2-3 engineers down to ~3-4 weeks.

PLAN-1's critical path (Phase 1 → 6 → 7) is correct. The parallel branches (2, 3, 4, 5) are correct in principle but Phase 5 needs the split.

H. Estimated Effort & Risk Hotspots

H.1 Per-phase effort estimates

Phase	Effort (1 engineer)	Notes
Phase 0 (hotfixes)	1-2 days	Build script fix + members API scope + testimonials gate
Phase 1 (schema + backfill + write-path)	3-5 days	Migration SQL + backfill scripts + 10 call-site fixes
Phase 2 (scope switcher + withScope)	3-5 days	New admin layout + ScopeSwitcher client component + 3 API routes + getCurrentUser refactor
Phase 3 (BrandAsset + Creative)	5-7 days	2 new models + resolver + 4 admin pages + 6 API routes + revalidateTag wiring
Phase 4 (email tier resolver)	5-7 days	resolveFromEmail + resolveStageTemplate + 8 file updates + relay-recipients wiring + IMAP round-robin
Phase 5a (timezone server-side)	3-5 days	chapter-tz.ts + datetime-tlv.ts refactor + 15 server component updates
Phase 5b (timezone client-side + UI copy)	5-7 days	30+ client component prop drilling + 30+ UI string replacements
Phase 6 (scope enforcement)	5-7 days	~30 API route migrations to withScope + canary logging + Socket.IO membership checks
Phase 7 (cleanup + URL aliases + i18n)	7-10 days	Event.chapter column drop audit + 2 new route segments + he-IL/fr-CA translations + RTL support
TOTAL	**~37-55 days**	~7-11 weeks sequential; ~4-6 weeks with 2-3 engineers parallel

H.2 Top 5 risk hotspots

1. **Build script `db push --accept-data-loss` fallback** (`package.json:7`). If a Phase 1 migration has ANY error, Vercel's build silently runs `db push --accept-data-loss` against production Neon — potentially dropping columns or resetting sequences. This is the single highest-risk item. MUST be fixed in Phase 0 before any migration lands.

2. **Phase 5 timezone migration is 5-10× bigger than estimated.** 177 occurrences across 43 files, most in CLIENT components that need prop drilling. If Phase 5 is attempted as a single PR, it will be unreviewable and likely ship with regressions (wrong timezone for edge cases like DST transitions). The split into 5a/5b is essential.

3. **EmailQueue / EventRsvp write-path gaps.** The orchestrator worker + flow-trigger + RSVP POST route all create rows without chapterId. If Phase 6 enforcement ships before these are fixed, chapter organizers will see empty lists (their own events' RSVPs and email queues will be invisible because the rows have NULL chapterId and the scope filter rejects them).

4. **Socket.IO mini-services have no scope enforcement.** `chat-service/index.ts:162` trusts the client on `chat:room:join`. A malicious user can join any room's socket and receive broadcasts. Phase 6 must add a server-side membership check, which requires modifying the mini-service (separate deployment, separate deploy cycle).

5. **`Event.chapter String` column drop in Phase 7.** Every reader of `event.chapter` (the legacy String property) must be migrated to `event.chapterRef.name` BEFORE the column is dropped. A grep audit for

`\.chapter\b` on Event instances is needed. If any reader remains, it throws at runtime. The build script's `db push --accept-data-loss` fallback (risk #1) compounds this — a malformed drop migration could succeed silently.